迴歸、聚類、神經網路

劉益宏
國立陽明交通大學電控所教授

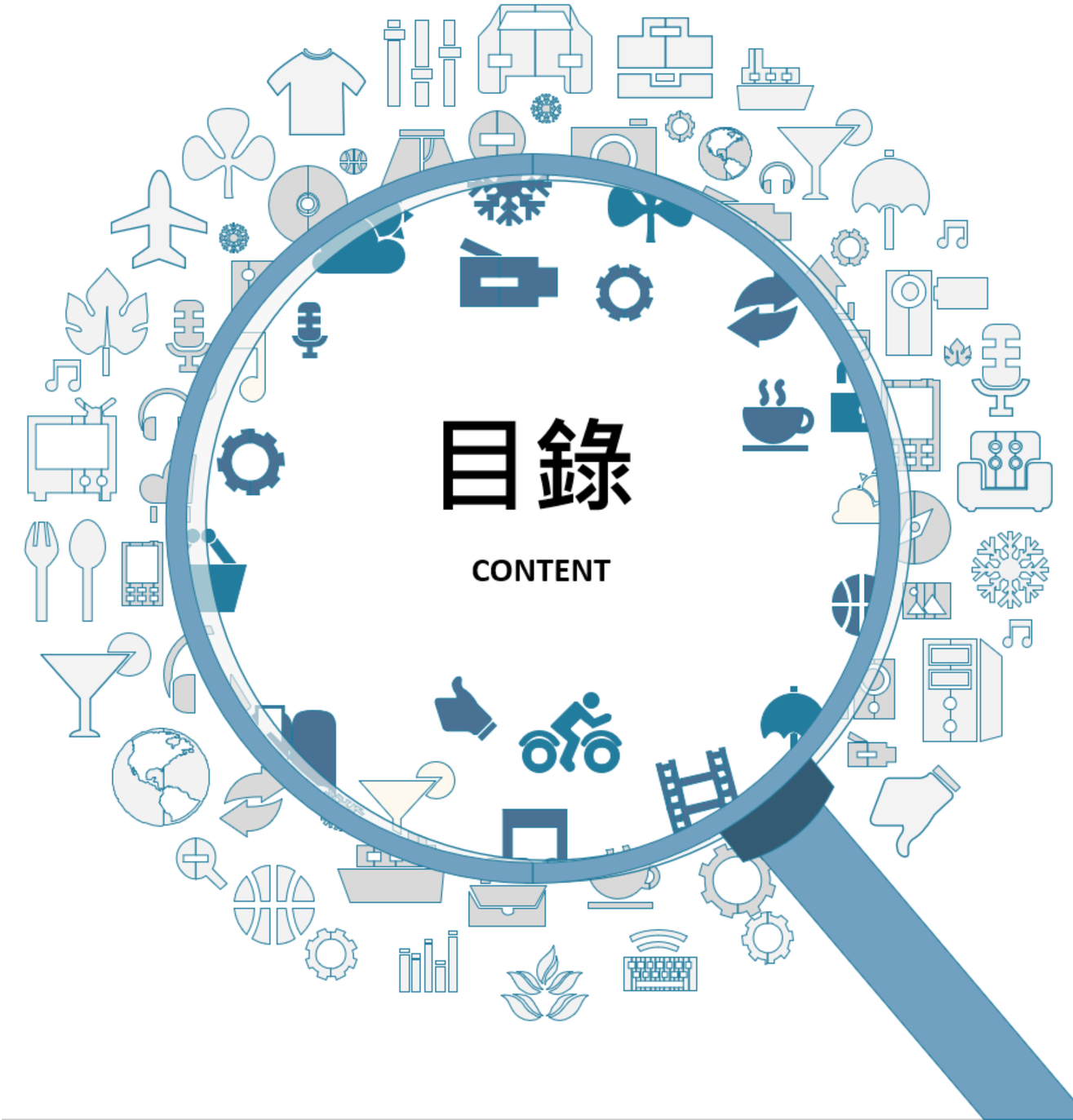
目錄

CONTENT

1 迴歸 Regression

2 聚類 Clustering

3 神經網路 Neural networks



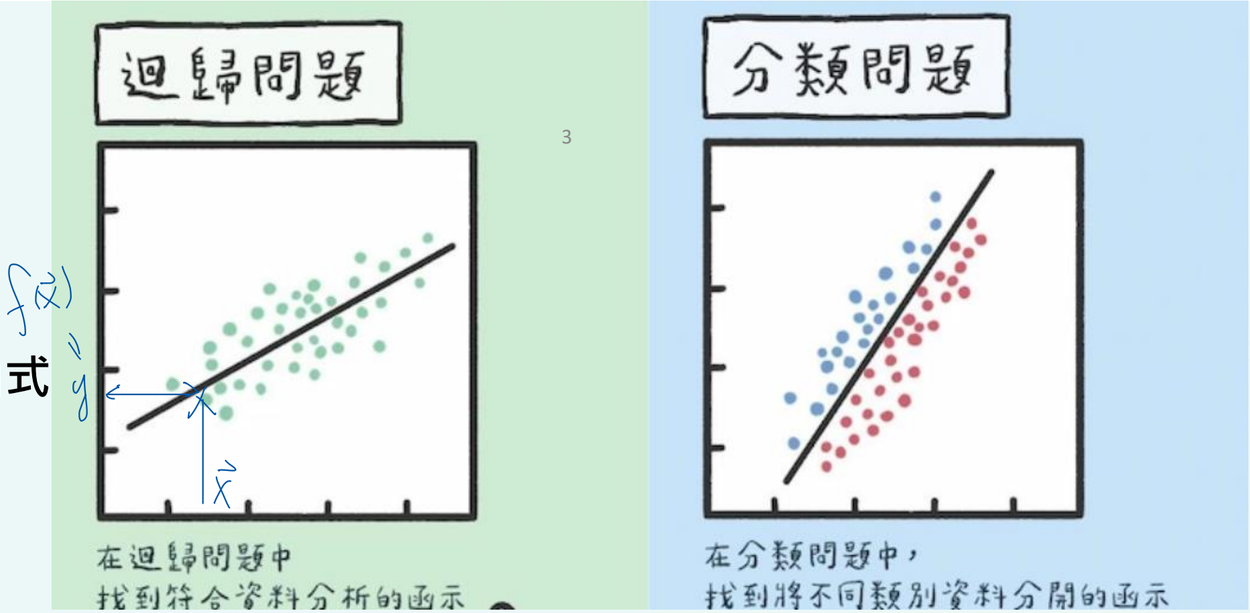
1. 迴歸與分類

- 在監督式學習中，分類（Classification）和回歸（Regression）是針對不同應用場景而使用的不同方法。主要差異在於預測的結果類型。

當你預測的目標輸出為離散的數值，稱其為分類(例如：判斷郵件是垃圾還是非垃圾)。

若為連續的數值，我們稱其為迴歸 (例如：預測股票價格或學生成績)。

找到資料分布規則的函式



找到分開資料的函式

1.迴歸與分類

Mapping function

- **迴歸的精神：**

1. 旨在理解和量化自變數（Independent variable）與應變數（Dependent variable）之間的關係。

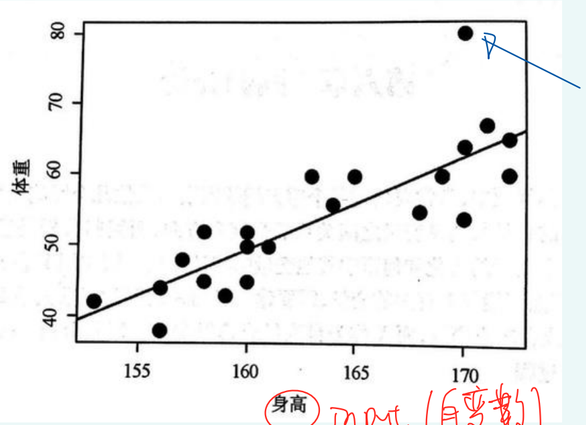
只有一個自變數和一個應變數的情形稱為簡單線性迴歸(Simple linear regression)， SISO

single input single output

大於一個自變數的情形稱為多元迴歸 (Multiple regression)。

多輸入單輸出 MISO

簡單線性回歸：



應變數

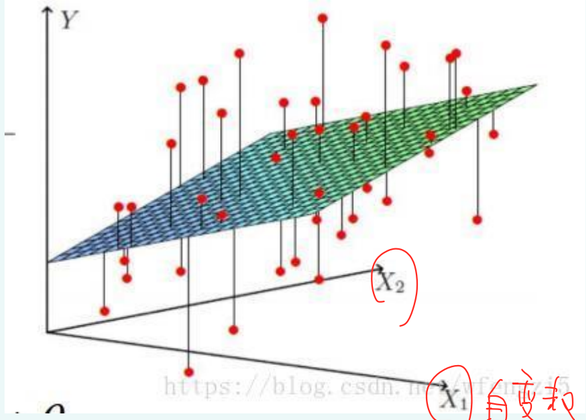
a sample (高重)已知

training sample

input (自變數)

假設自變數=身高=x，應變數=體重=y

二元線性回歸：



$X_1, X_2 \rightarrow \boxed{MR} \rightarrow y$

自變數

有兩個自變數x1、x2

2.迴歸模型能夠捕捉變數之間的趨勢和關聯性。

兩個都是 linear model

2. 簡單線性迴歸 (Simple linear regression)

- 實例推導：

實際體重值

假設有 n筆資料 $(x_i, y_i), i = 1, \dots, n$，橫軸為身高，縱軸體重，預求這兩變數所形成的**迴歸**方程式。

$f(x_1) = \hat{y}_1 \dots$ 預測值

$f(x_2) = \hat{y}_2$

$\vdots$

$f(x_n) = \hat{y}_n$

讓誤差平方值

Overall error

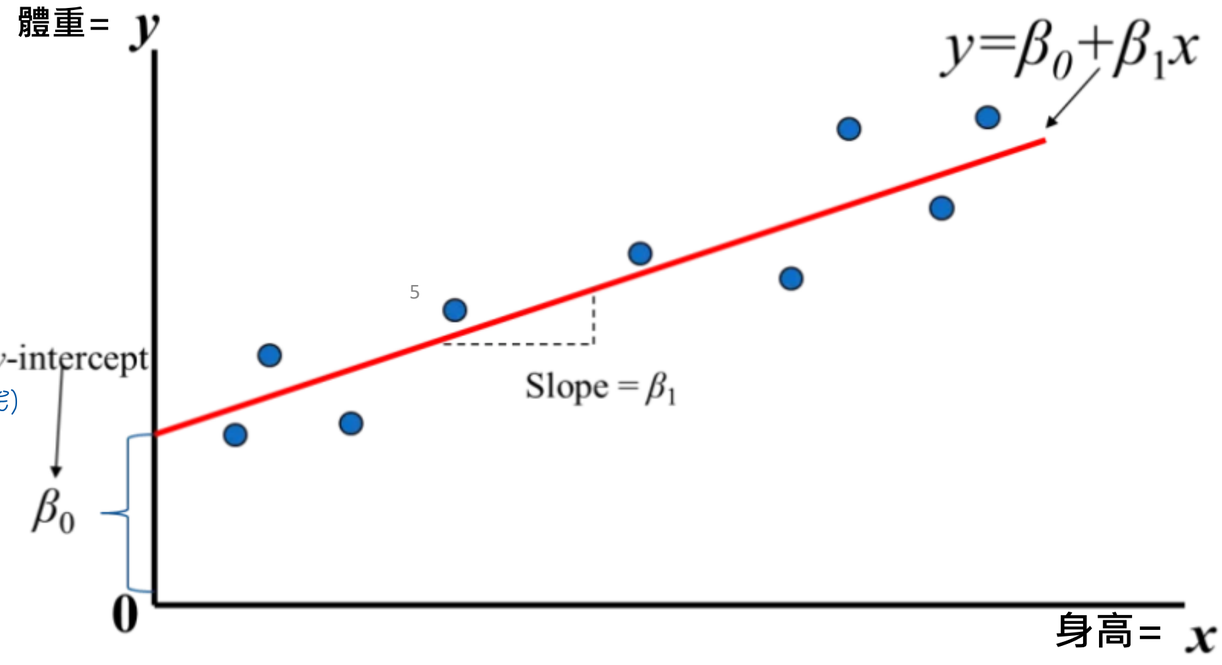
$$\frac{[(\hat{y}_1 - y_1)^2 + \dots\dots + (\hat{y}_n - y_n)^2]}{n} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

mean square error (MSE)

Output

簡單線性回歸可以寫成

$y = \beta_0 + \beta_1 x$



input

β_0：截距(Intercept)

β_1：斜率(Slope)

n ：資料總數

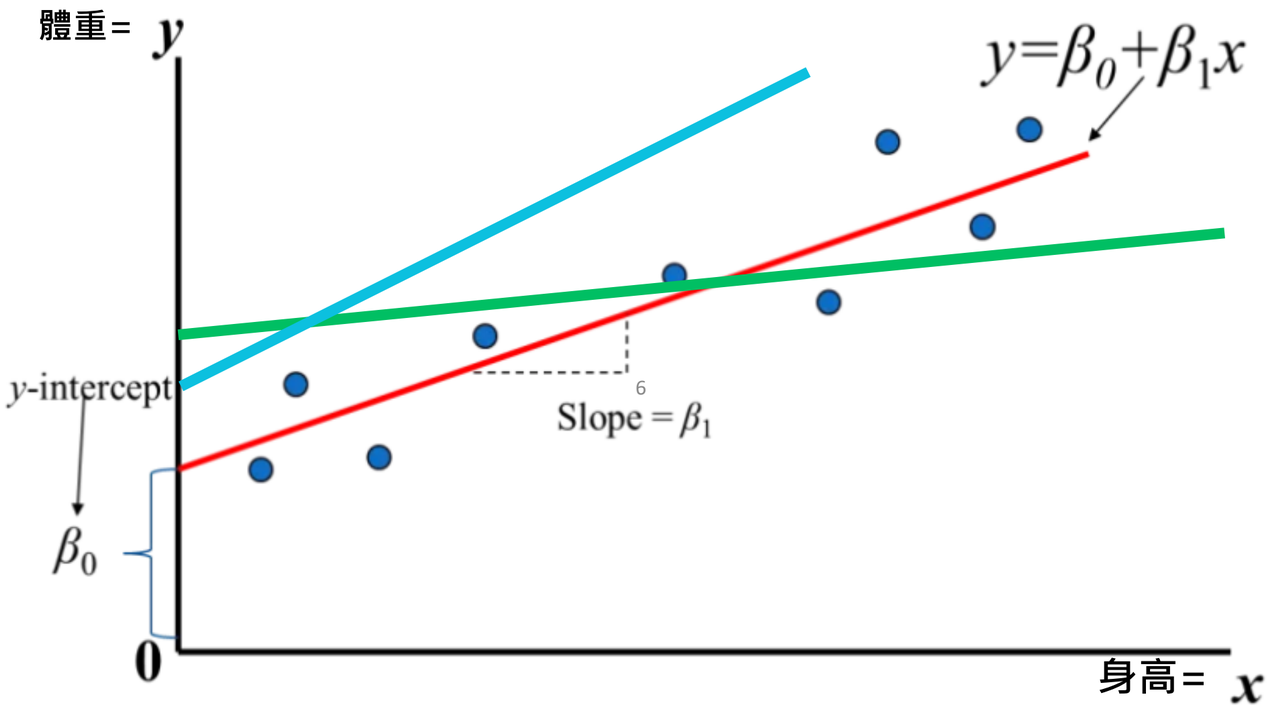
2. 簡單線性迴歸 (Simple linear regression)

β_0：截距(Intercept)
β_1：斜率(Slope)
n ：資料總數

- 但這迴歸方程式有很多條，如何找到最好、最有代表性的迴歸方程呢？

簡單線性回歸可以寫成

$y=\beta_0+\beta_1 x$ → min MSE



也就是如何找到最好的$\boldsymbol{\beta_0}$ 、 $\boldsymbol{\beta_1}$？

2. 簡單線性迴歸(Simple linear regression)

- 先定義$\hat{y}_i$：將n 個資料點, $(x_i, y_i), i = 1, \ldots, n$都代入**迴歸**方程式內，是為預估的體重。

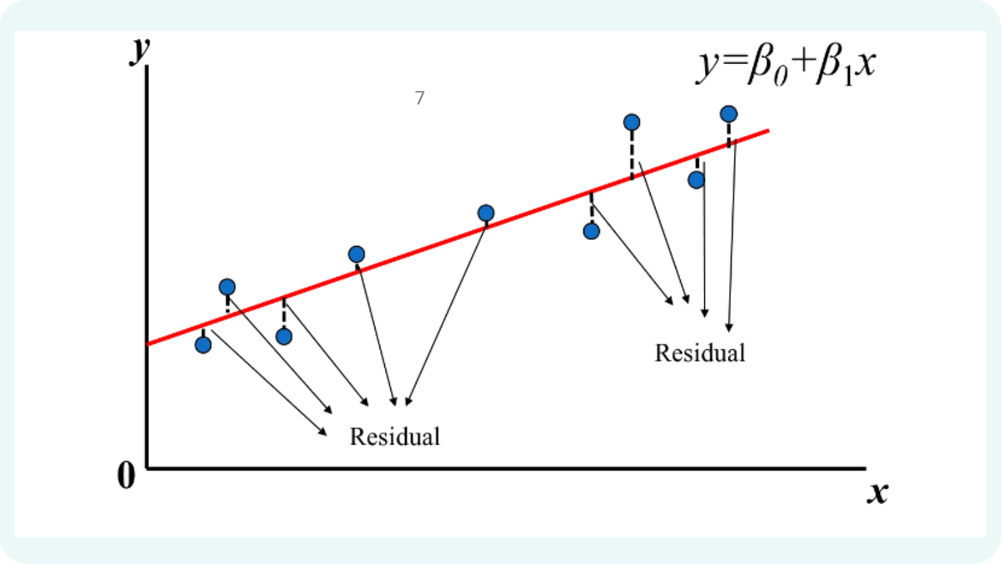
predicted value

$$\hat{y}_i = \beta_0 + \beta_1 x_i \ , i = 1, \ldots, n$$

- 再定義ε_i： 預估出來的體重和真實資料還是會有誤差(error)或稱為殘差(Residual)。

$$\varepsilon_i = y_i - \hat{y}_i$$

actual value



2. 簡單線性迴歸 (Simple linear regression)

- 迴歸分析的目標函數/損失函數(loss function)就是希望找到的模型最終的殘差越小越好，統計上會用一個名詞: 最小平方法(Least Square)來找參數($\boldsymbol{\beta_0}$ 、 $\boldsymbol{\beta_1}$)。

$\because \hat{y}_i = \hat{\beta}_0 - \hat{\beta}_1 x_i$

Loss是β_0, β_1的函數

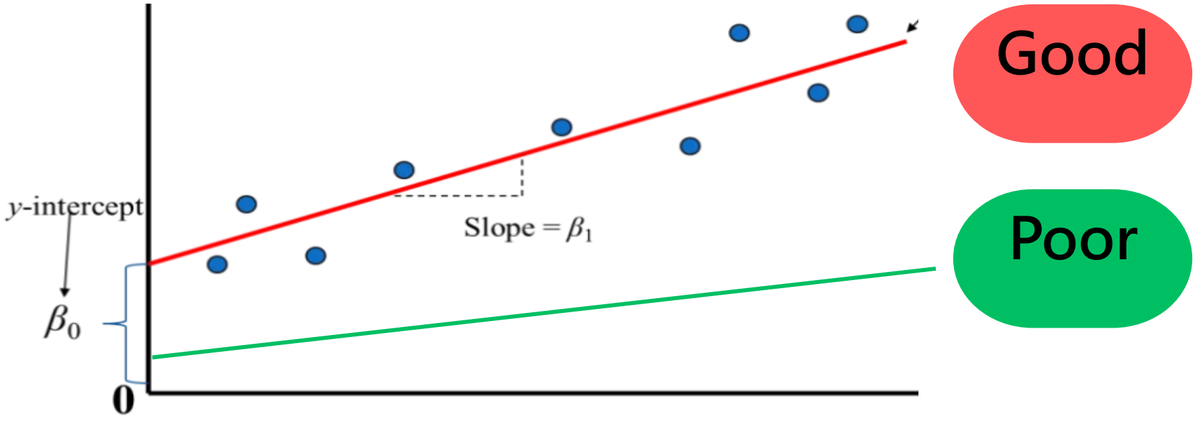
$$loss(\hat{\beta}_0, \hat{\beta}_1) = \underbrace{\boxed{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}}_{SE} = \sum_{i=1}^{n}(\overset{\text{known}}{y_i} - (\underbrace{\hat{\beta}_0 + \hat{\beta}_1 \overset{\text{known}}{x_i}}_{\text{unknown}}))^2$$

unknown → variable

因此我們希望訓練樣本的誤差平方和(Sum Square error, SSE)越小越好。

① $\frac{\partial Loss}{\partial \beta_0} = 0$

② $\frac{\partial Loss}{\partial \beta_1} = 0$



2. 簡單線性迴歸 (Simple linear regression)

- 為了推估 β_0，對 $loss(\beta_0, \beta_1)$做β_0偏微分等於0

$$loss(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n}(y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2$$

$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ (所有實際輸出值的平均)

$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$

$$\boxed{\frac{\partial loss(\hat{\beta}_0, \hat{\beta}_1)}{\partial \beta_0}} = \frac{\partial \sum_{i=1}^{n}(y_i - \hat{\beta}_0 + \hat{\beta}_1 x_i)^2}{\partial \beta_0} = 0$$

$$\rightarrow \ -2\sum_{i=1}^{n}(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0$$

$$\rightarrow \ \hat{\beta}_0 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\beta}_1 x_i)$$

$$\rightarrow \ \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad \boxed{1}$$

2. 簡單線性迴歸 (Simple linear regression)

- 為了推估 β_1，對 $loss(\beta_0, \beta_1)$做β_1偏微分等於0

$$loss(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n}(y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2$$

$$\frac{\partial loss(\hat{\beta}_0, \hat{\beta}_1)}{\partial \beta_1} = \frac{\partial \sum_{i=1}^{n}(y_i - \hat{\beta}_0 + \hat{\beta}_1 x_i)^2}{\partial \beta_1} = 0$$

$$\rightarrow \ -2\sum_{i=1}^{n}(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)x_i = 0$$

$$\rightarrow \ \sum_{i=1}^{n}(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)x_i = 0$$

$$\rightarrow \ \sum_{i=1}^{n} y_i x_i - \sum_{i=1}^{n} \hat{\beta}_1 x_i^2 - \underline{\sum_{i=1}^{n} \hat{\beta}_0 x_i = 0} \quad 1$$

$$\rightarrow \ \sum_{i=1}^{n} y_i x_i - \hat{\beta}_1 \sum_{i=1}^{n} x_i^2 - \underline{\sum_{i=1}^{n}(\bar{y} - \hat{\beta}_1 \bar{x})x_i} = 0$$

$$\rightarrow \ \sum_{i=1}^{n} y_i x_i - \hat{\beta}_1 \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} \bar{y} x_i + \sum_{i=1}^{n} \hat{\beta}_1 \bar{x} x_i = 0$$

2. 簡單線性迴歸 (Simple linear regression)

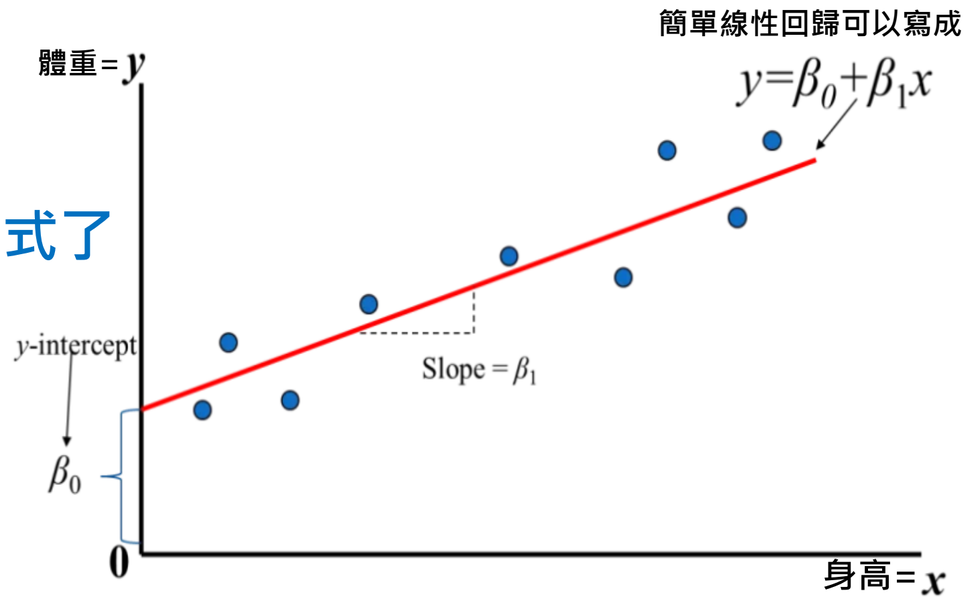
$$\rightarrow \sum_{i=1}^{n} y_i x_i - \hat{\beta}_1 \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} \bar{y}\, x_i + \sum_{i=1}^{n} \hat{\beta}_1 \bar{x} x_i = 0$$

$$\rightarrow \hat{\beta}_1 \sum_{i=1}^{n} (x_i - \bar{x}) x_i = \sum_{i=1}^{n} (y_i - \bar{y}) x_i$$

$$\rightarrow \boxed{\hat{\beta}_1 = \frac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{n} (x_i - \bar{x})^2}}$$

11

- 求得β_0、β_1就可以推出這條簡單線性迴歸的方程式了



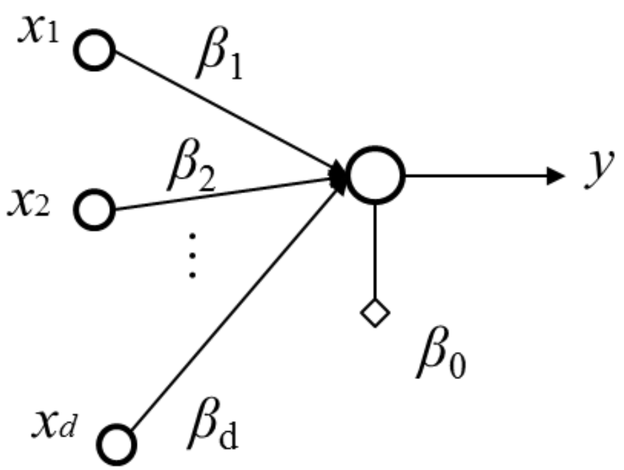
3. 多元線性迴歸 (Multiple linear regression)

β_0：截距(Intercept)
$\beta_1 \ldots \beta_k$ ：斜率(Slope)
n ：資料總數

- 多元線性迴歸跟前面簡單線性回歸基本上一樣，只是多了幾個自變數，因此多元回歸的自變數(x)，可以用矩陣的方式(x)來推導，基本上和簡單線性迴歸大同小異。

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

12



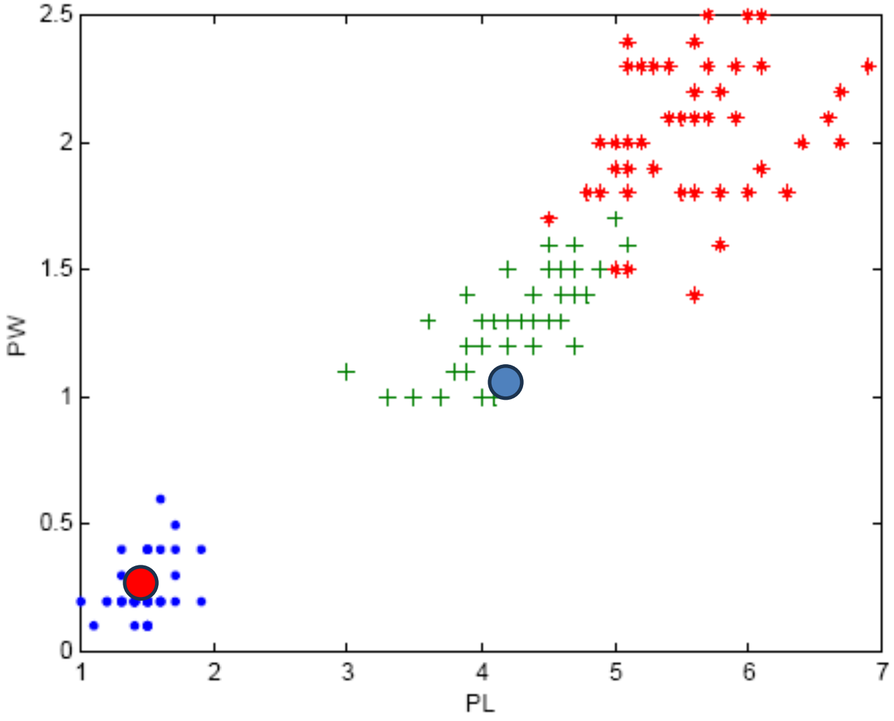
- 這結構似乎很像只有一個輸入層和一個輸出層的神經網路，所以可以用SGD去找β的最佳解。

隨機梯度下降法(Stochastic gradient descent, SGD)

待補多元現行回歸的方程式

聚類 Clustering

- Basic concept : 以IRIS DATA為例說明



K=2

The symbols ".", "+", and "*" represent the classes *Iris setosa* (class 1), *Iris versicolor* (class 2), and *Iris virginica* (class 3), respectively

Clustering algorithm：*K*-means algorithm

K-means algorithm

- Given a set of unlabeled data points (X1, X2, …., Xn)
- Initialize the number of clusters *k* (to cluster the data points into *k* groups), where *k* = 2, 3, ….., n. (a user-defined parameter)
- Step 1 : initialize the centers of the *k* clusters randomly (利用亂數產生k個座標，當作這k個clusters的初始中心座標)
- Step 2 : assign the data points to the clusters based on their distance to the cluster centers (把每一筆data歸類到其中的任何一個cluster/group，根據min distance criterion)
- Step 3: re-compute the centroids (centers) of the *k* clusters
- Step 4: go to 2nd step until convergence (新的cluster centers 和上一次的cluster centers不再有所差距)

K-means Algorithm - discussion

- Result can vary significantly based on initial choice of seeds
- Algorithm can get trapped in a local minimum
  - Example : four data points at the vertices of a two-dimensional rectangle
    - Local minimum : two cluster centers at the midpoints of the rectangles long sides
- Simple way to increase chance of finding a global optimum : restart with different random seeds

Fuzzy c-means (FCM) algorithm:

another data clustering method

- Basic idea:

Fuzzy c-means (FCM) algorithm: formulation

- Given a dataset $\{\mathbf{x}_i \in \boldsymbol{R}^d\}_{i=1}^N$, (FCM) algorithm is to partition the dataset into disjoint clusters by solving the optimization problem,

$$\begin{cases} \textit{Minimize} \quad J(T;\boldsymbol{U};\boldsymbol{V}) = \sum_{j=1}^{c}\sum_{i=1}^{N} u_{ji}^m \|\mathbf{x}_i - \mathbf{v}_j\|^2 \\ \textit{subject to the constraints,} \\ \sum_{j=1}^{c} u_{ji} = 1, \ \forall i = 1,\ldots,N, \\ 0 < \sum_{i=1}^{N} u_{ji} < N, \ \forall j = 1,..,c \end{cases}$$

where

- $u_{ji} \in [0,1]$ is the membership degree that the ith data point $\mathbf{x}_i$ belongs to the jth cluster
- $\boldsymbol{U} = [u_{ji}]$ is the partition matrix
- $\mathbf{v}_j \in R^d$ is the centroid of the jth cluster
- $\mathbf{V} = (\mathbf{v}_1,\ldots,\mathbf{v}_j,\ldots,\mathbf{v}_c)$ is a c-tuple of cluster prototypes
- c is the number of clusters ($2 \le c \le N$)
- $m \in (1,\infty)$ is the weight controlling the degree of fuzziness of the matrix $\boldsymbol{U}$ (m is set as 2 in general)

update

$\mathbf{v}_j$ by

Fuzzy c-means (FCM) algorithm

– learning steps

- Step 1 : *initialize the number of clusters c*
- Step 2 : *initialize* $\mathbf{v}_j$ *by random*
- Step 3 : *compute* u_{ji} *by* $u_{ji} = \frac{\left(1/\|\mathbf{x}_i - \mathbf{v}_j\|^2\right)^{1/(m-1)}}{\sum_{k=1}^{c}(1/\|\mathbf{x}_i - \mathbf{v}_k\|^2)^{1/(m-1)}}$
- Step 4 : *update the cluster centroids* $\mathbf{v}_j$ *by* $\mathbf{v}_j = \frac{\sum_{i=1}^{N} u_{ji}^m \mathbf{x}_i}{\sum_{i=1}^{N} u_{ji}^m}$, $j = 1, .., c$
- Step 5 : *repeat step 3 and step 4 until* $\|\boldsymbol{U}\| < \varepsilon$, *where* ε is *a small positive value.*

Fuzzy c-means (FCM) algorithm

– to determined the optimal (most plausible) number of clusters

- Choosing a proper number c of clusters is of primary importance. The best number c_{best} of clusters can be determined using the paritioning entropy (分割熵)-based criterion

$$c_{best} = \arg\min_{c=2}^{N}\{\min_{\mathbf{U}\in\mathbf{\Omega}_c}[H(\mathbf{U},c)]\}$$

where $\boldsymbol{\Omega}_c$ is the set of solutions, and $H(\boldsymbol{U},c) = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{c}\left|u_{ji}\, ln\, u_{ji}\right|$

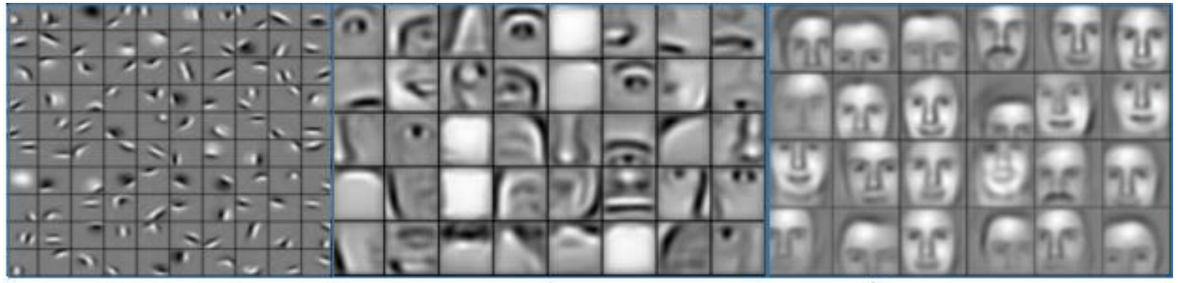
is the paritioning entropy, providing a global validity measure for the clustering results.

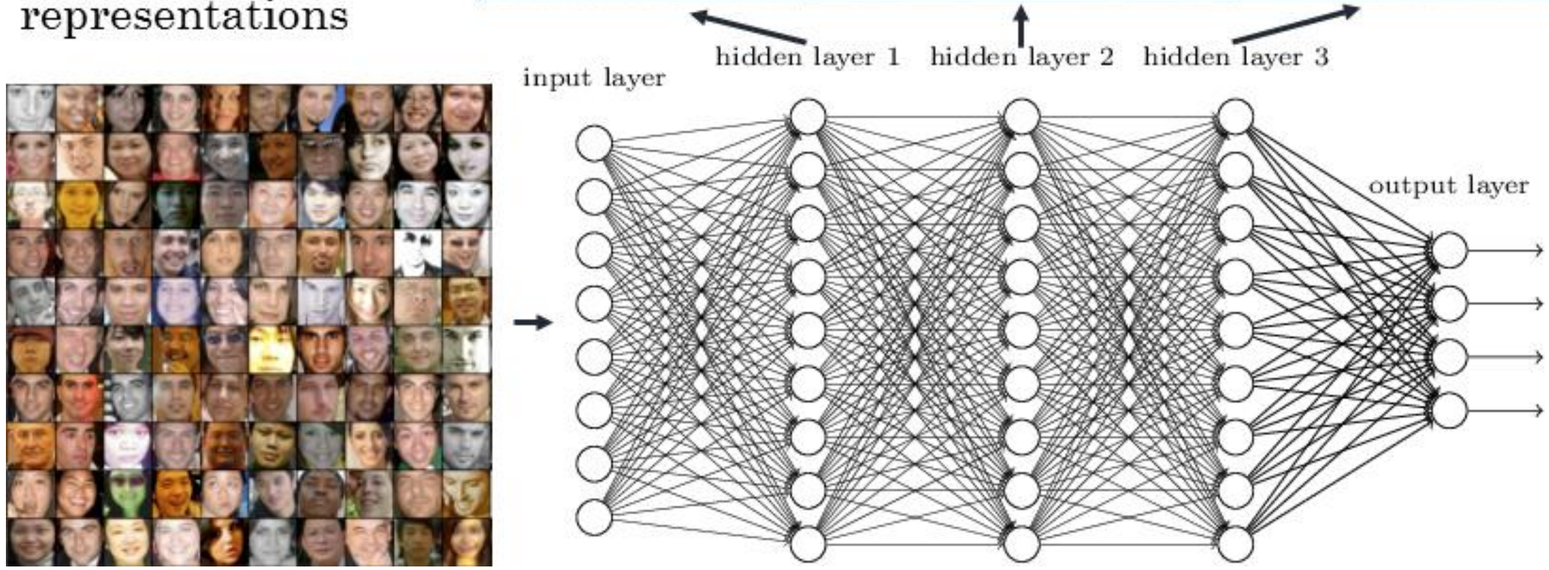
- Finally, the dataset T is partitioned into c_{best} subsets using the simple rule : the ith training data point $\boldsymbol{x}_i$ is assigned to the kth cluster if its membership degree to the kth cluster is the highest, namely

$$k = \arg\max_{j=1}^{c_{best}} u_{ji}$$

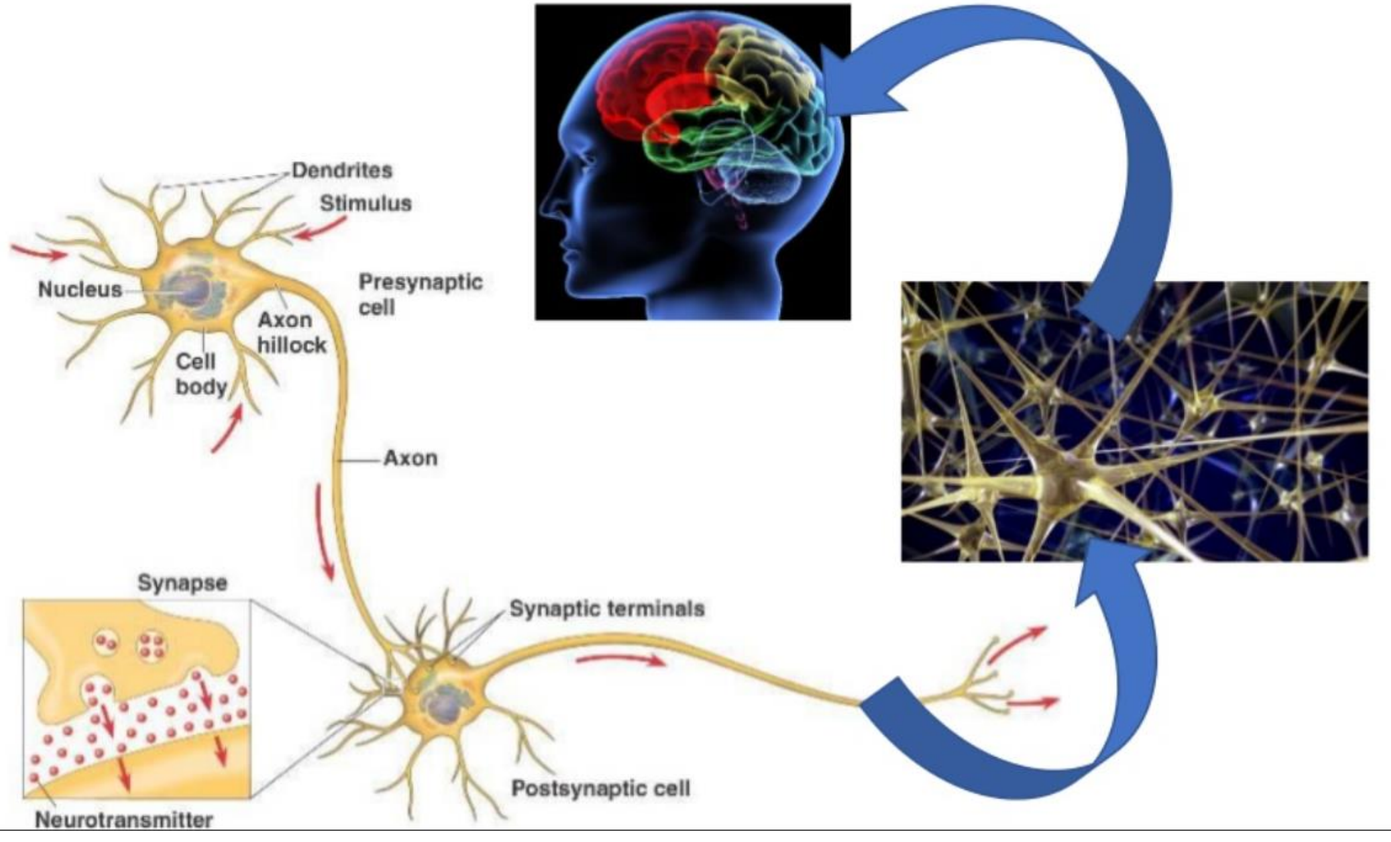
Introduction to
Neural Network (DNN)
神經網路

Deep neural networks learn hierarchical feature representations





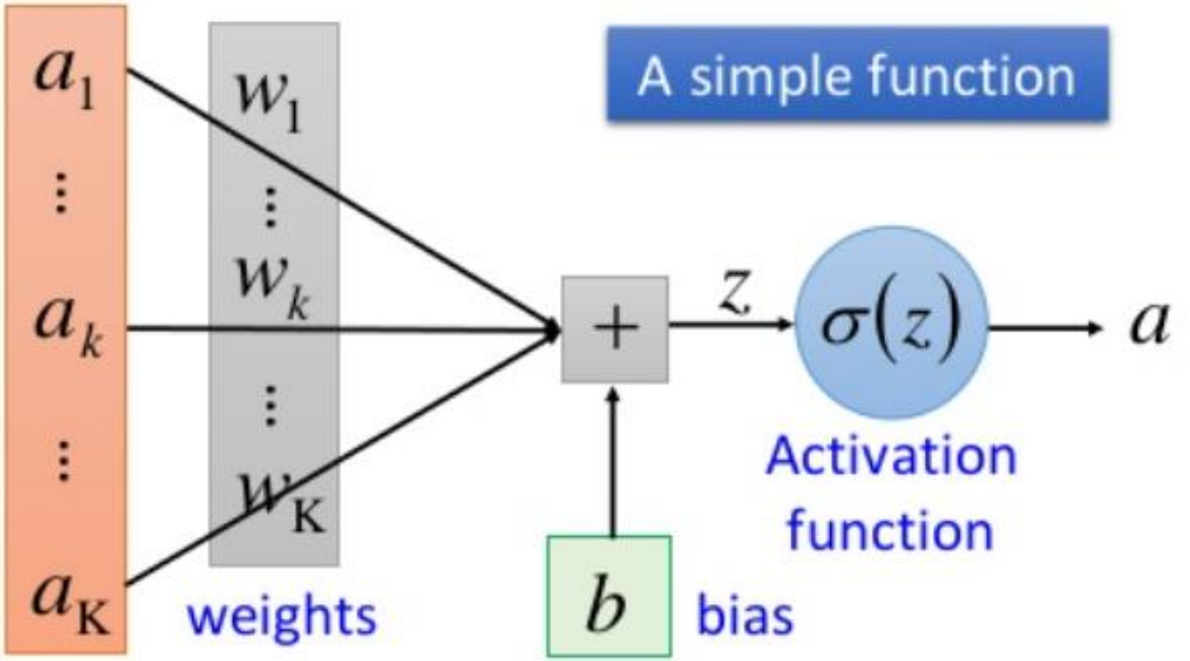
Human Brains



Neural Network

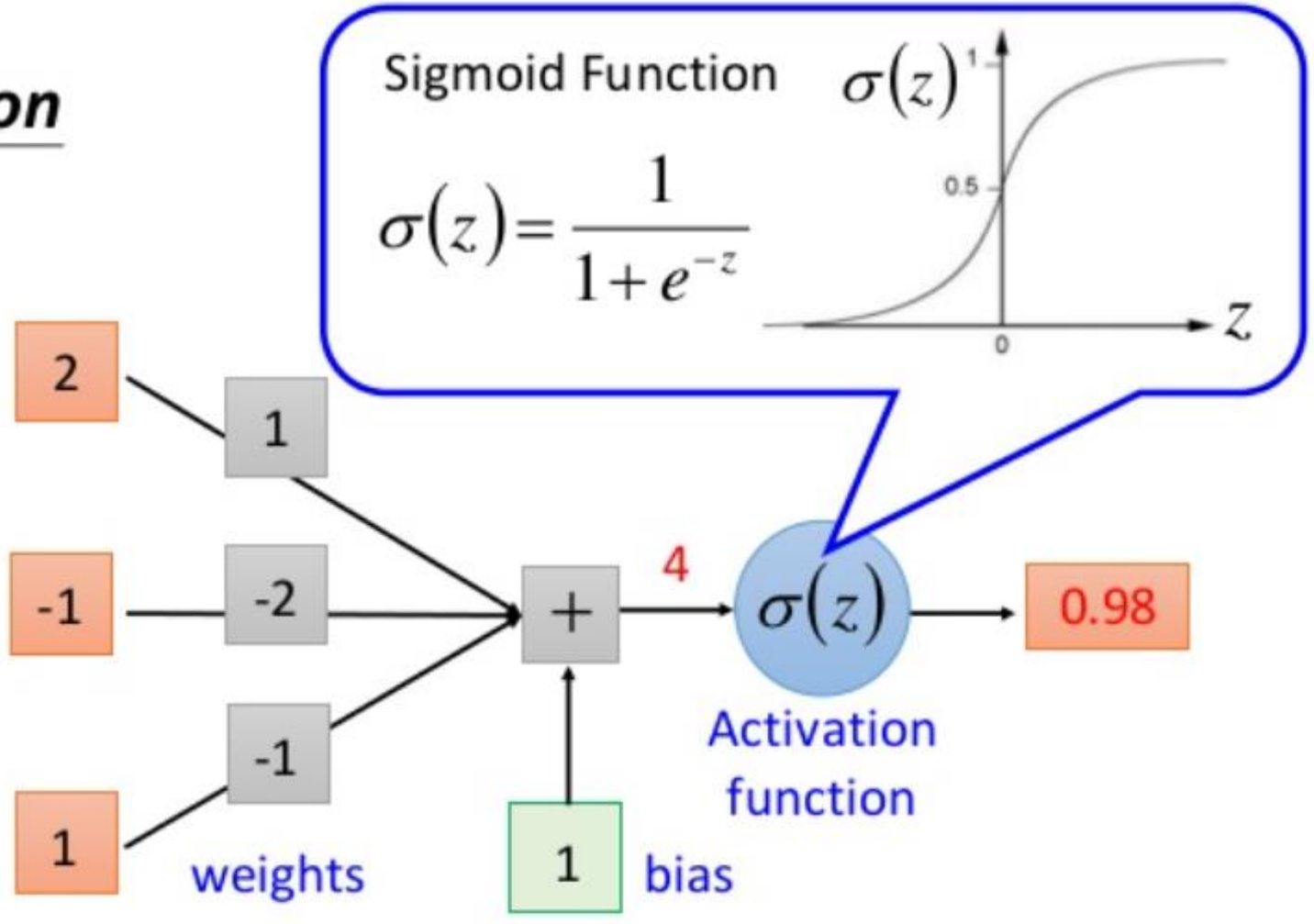
Neuron

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$



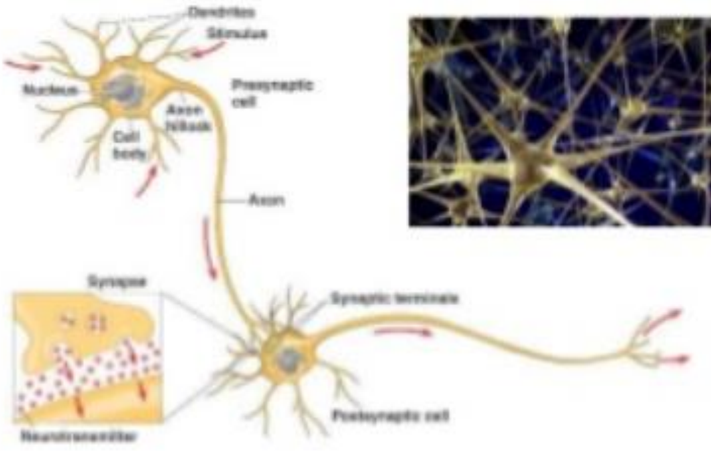
Neural Network

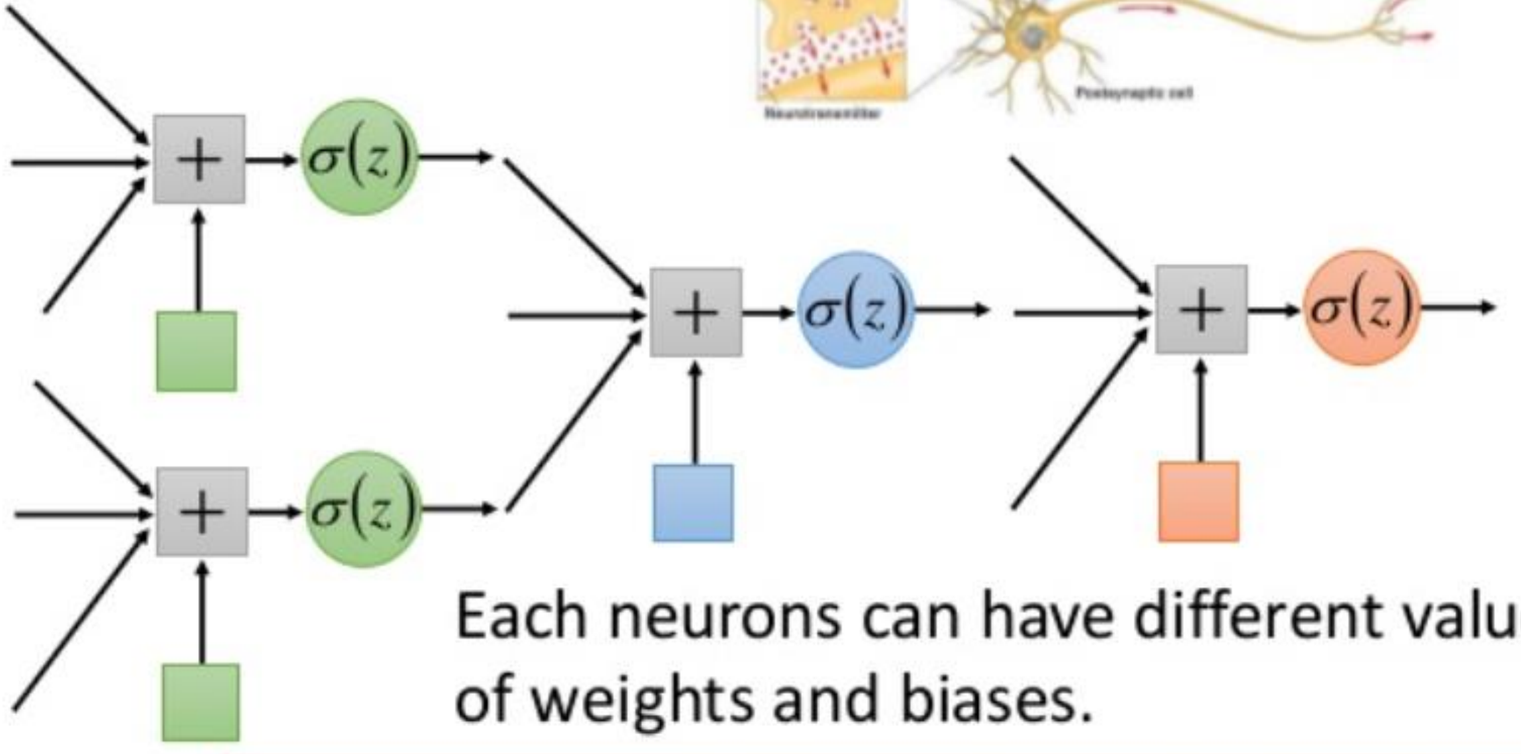
Neuron



Neural Network

Different connections leads to different network structure

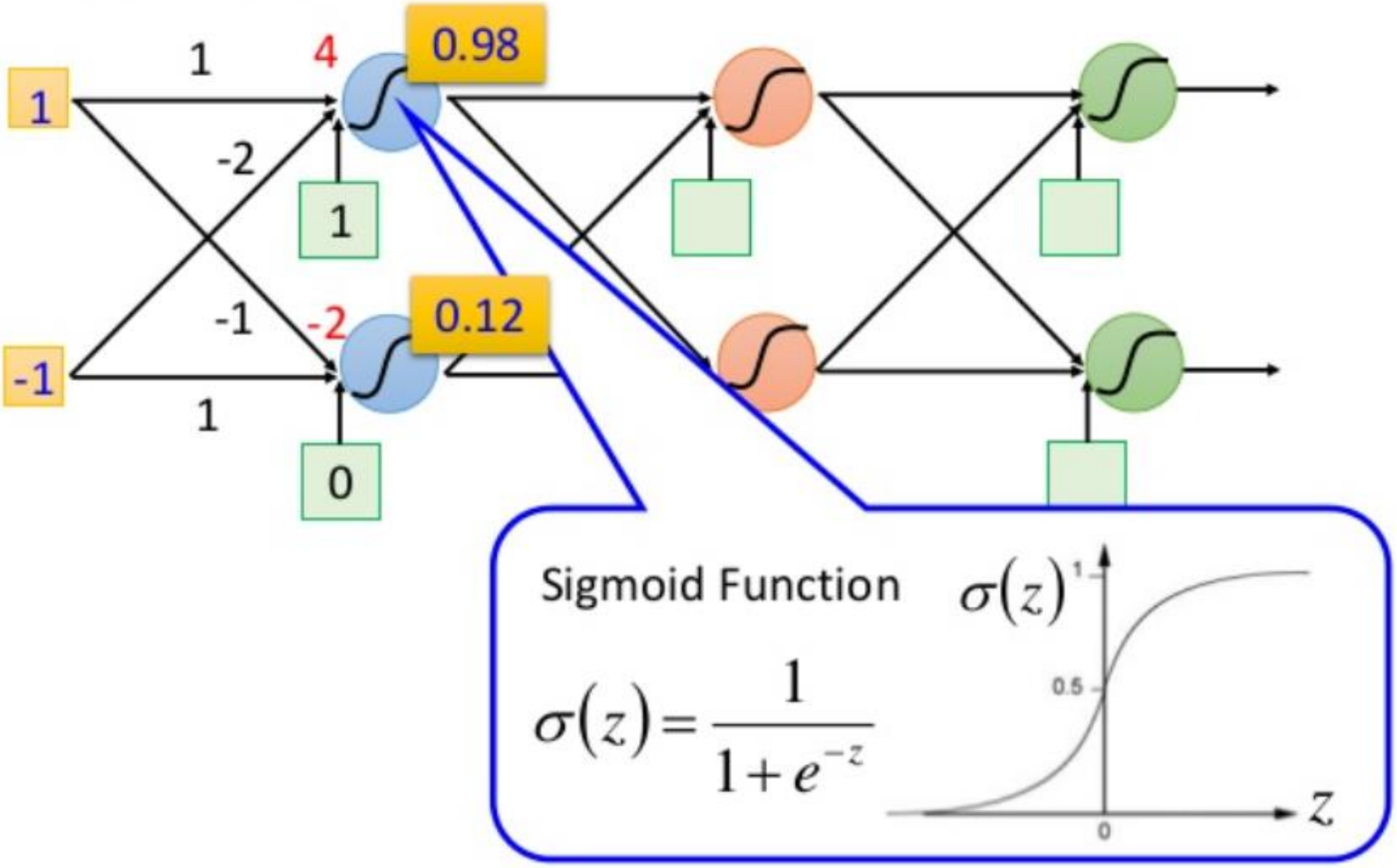




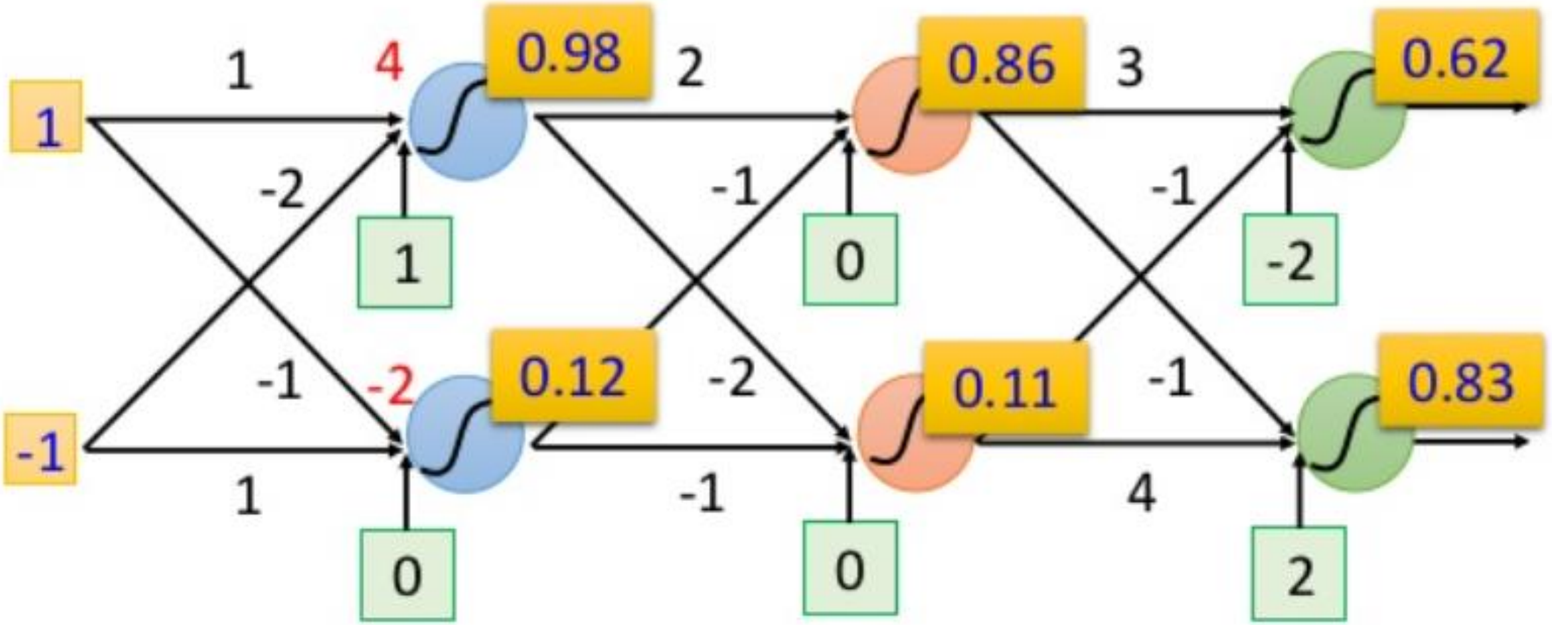
Each neurons can have different values of weights and biases.

Weights and biases are network parameters θ

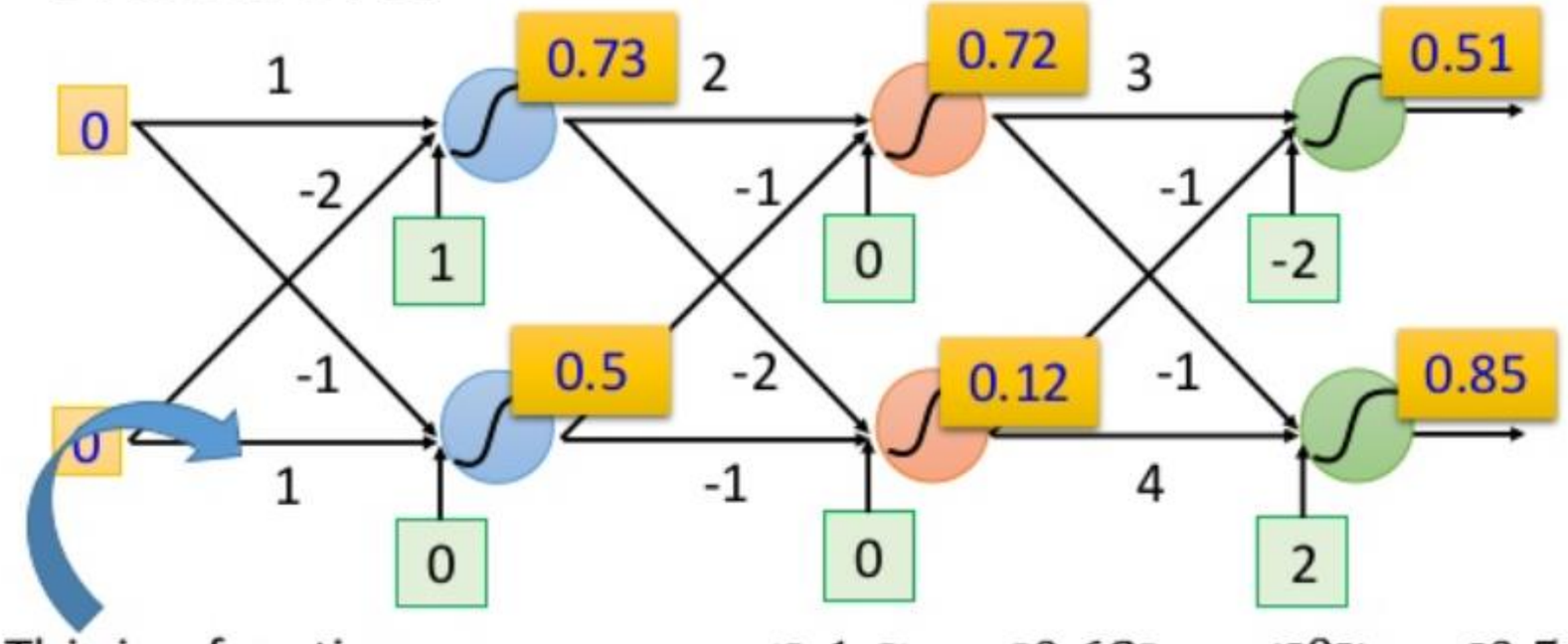
Fully Connect Feedforward Network



Fully Connect Feedforward Network



Fully Connect Feedforward Network



This is a function.

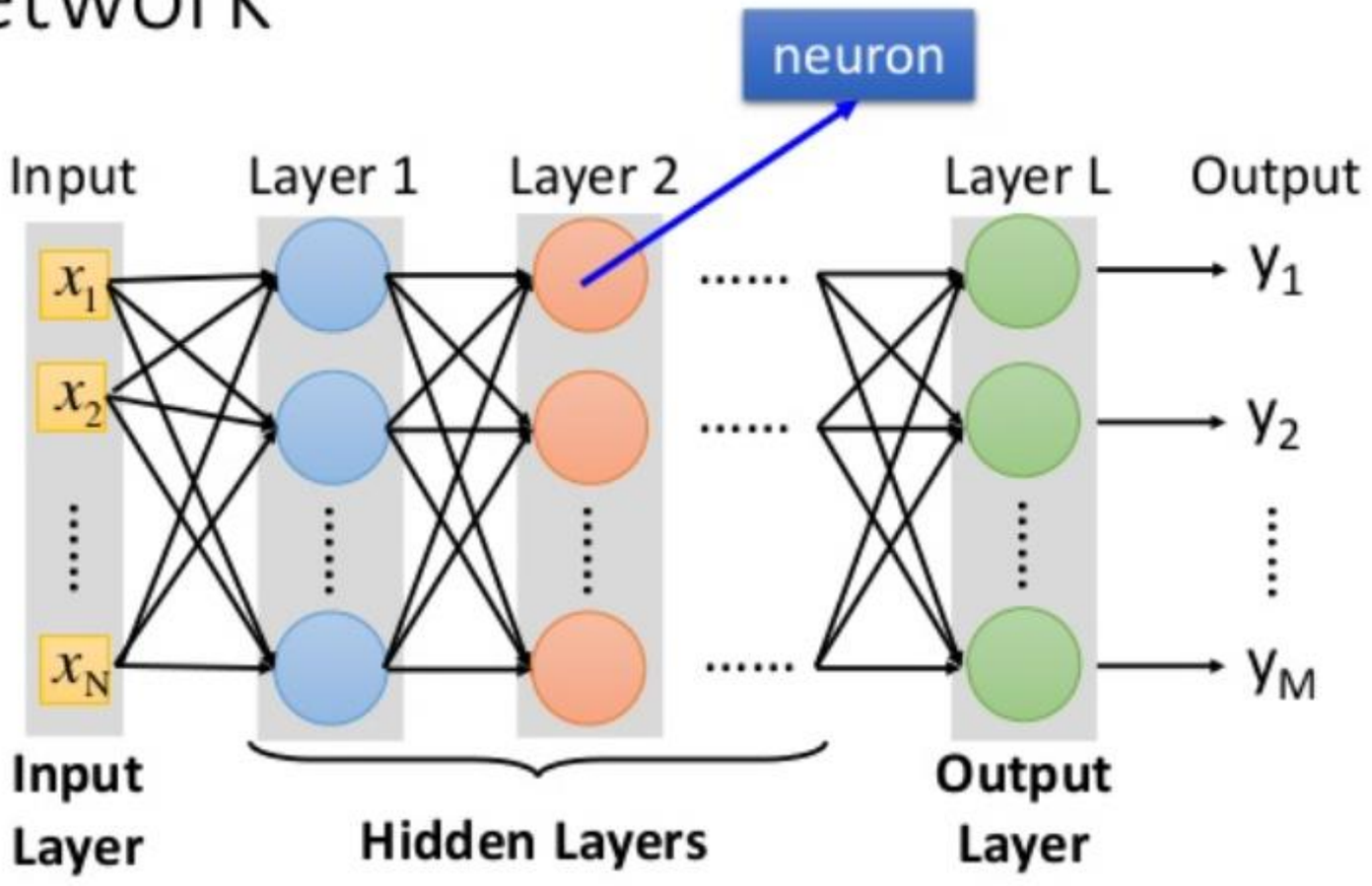
Input vector, output vector

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix} \quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

Given parameters θ, define a function

Given network structure, define ***a function set***

Fully Connect Feedforward Network

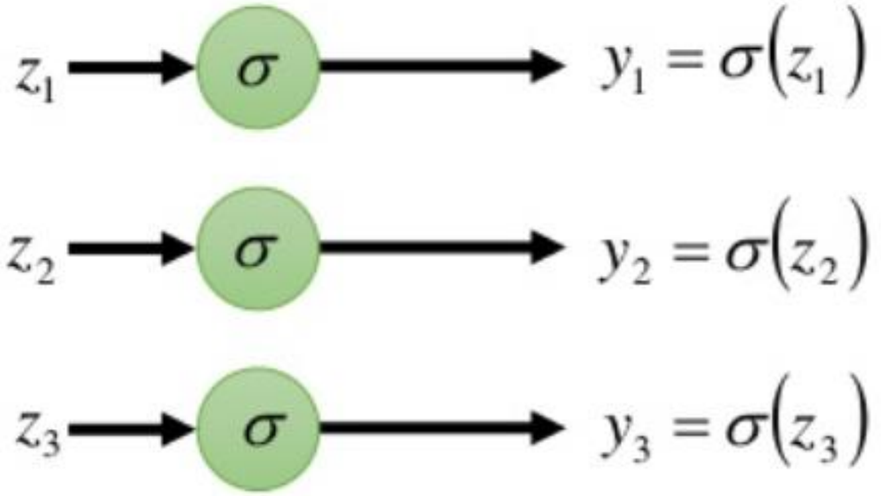


Deep means many hidden layers

Output Layer (Option)

- Softmax layer as the output layer

Ordinary Layer



In general, the output of network can be any value.

May not be easy to interpret

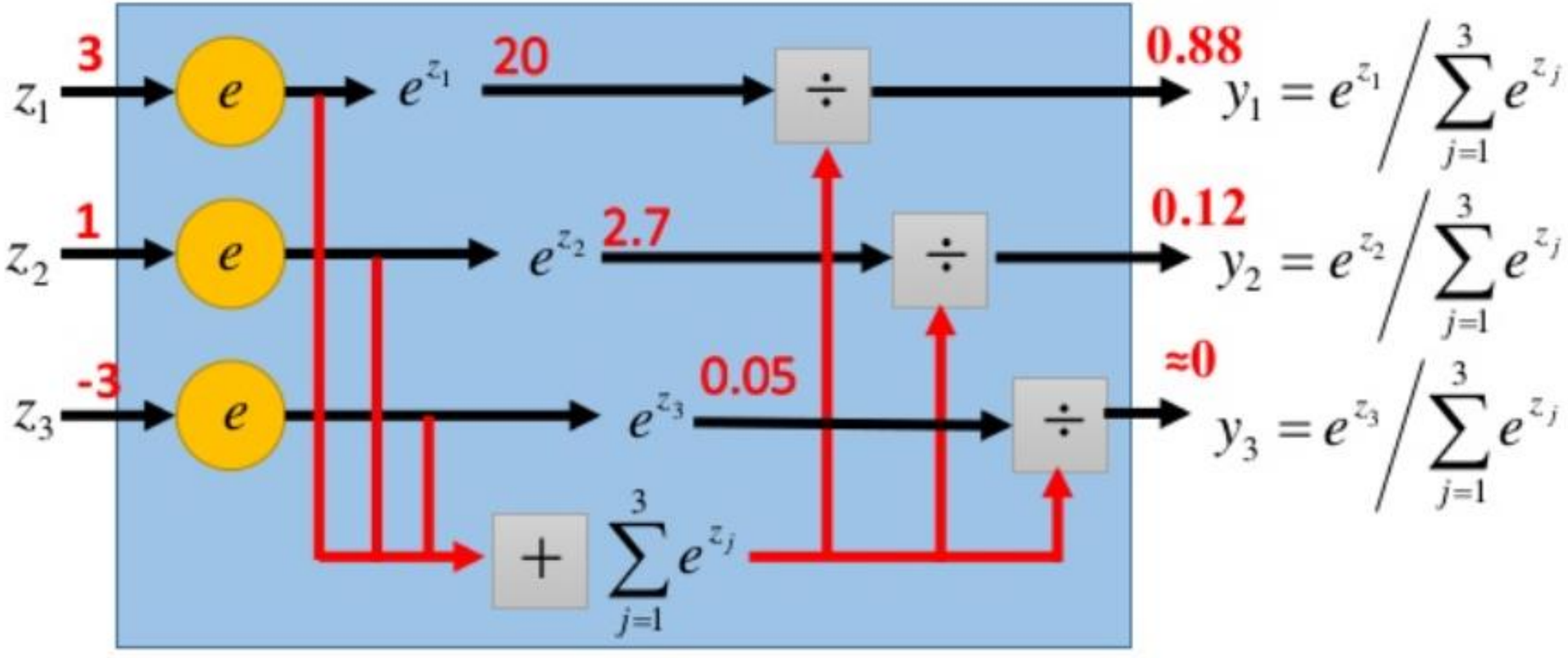
Output Layer (Option)

- Softmax layer as the output layer

Probability:

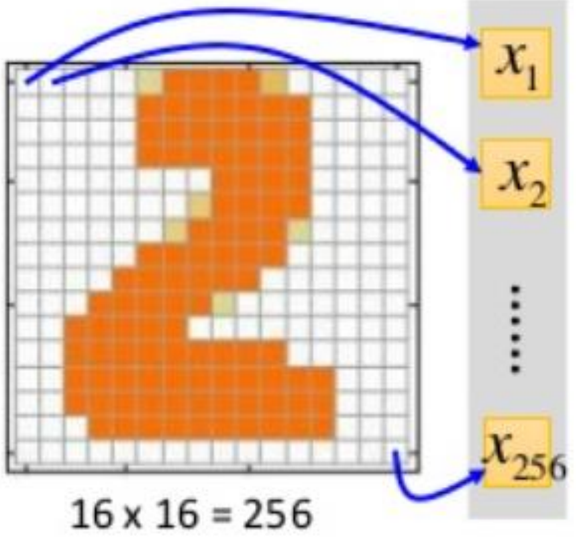
- $1 > y_i > 0$
- $\sum_i y_i = 1$

Softmax Layer



Example Application

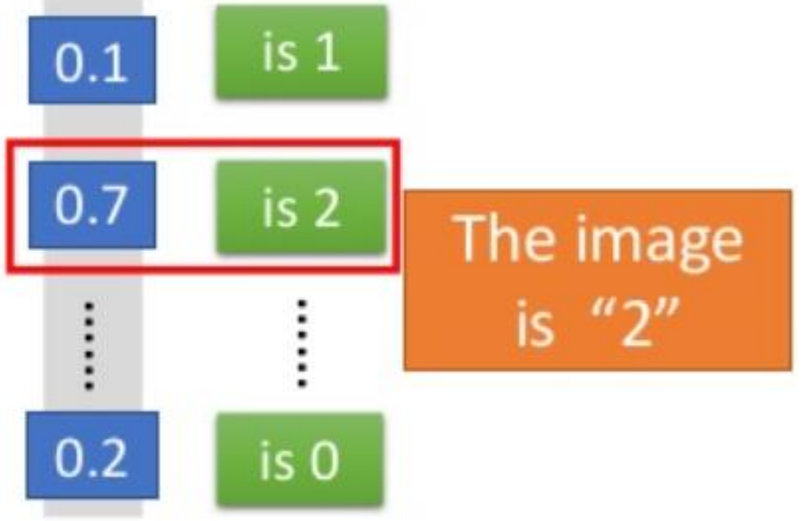
Input



Ink → 1

No ink → 0

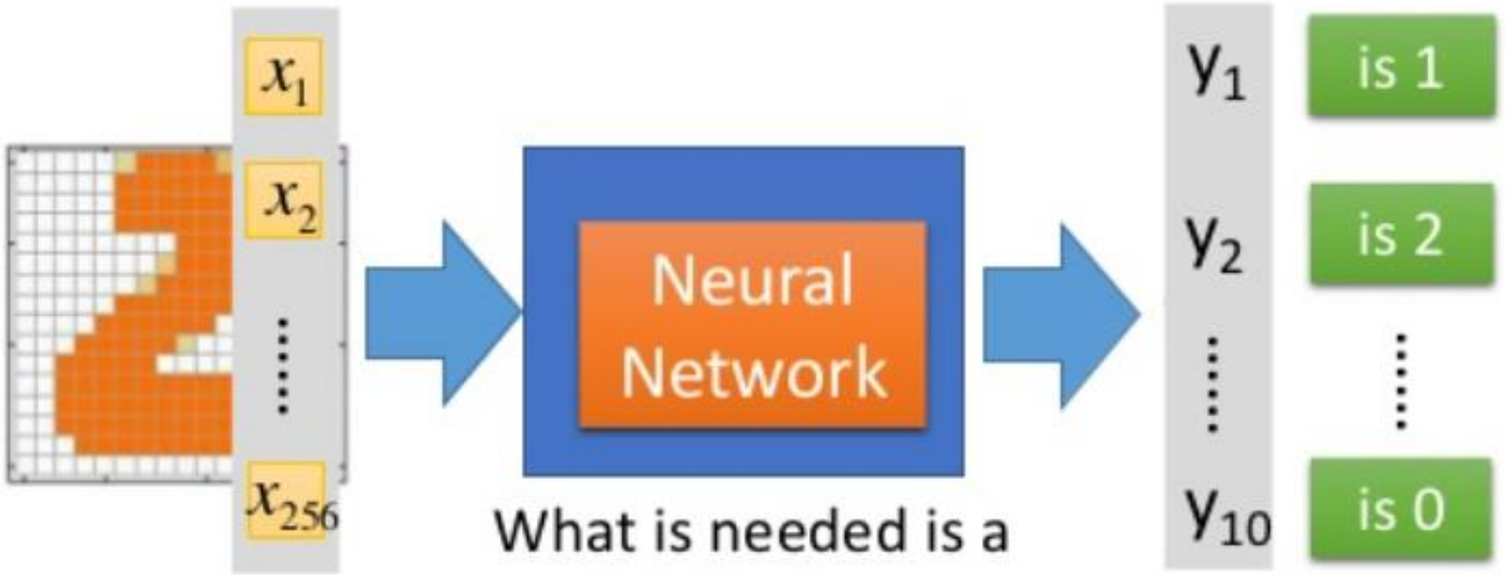
Output



Each dimension represents the confidence of a digit.

Example Application

- Handwriting Digit Recognition

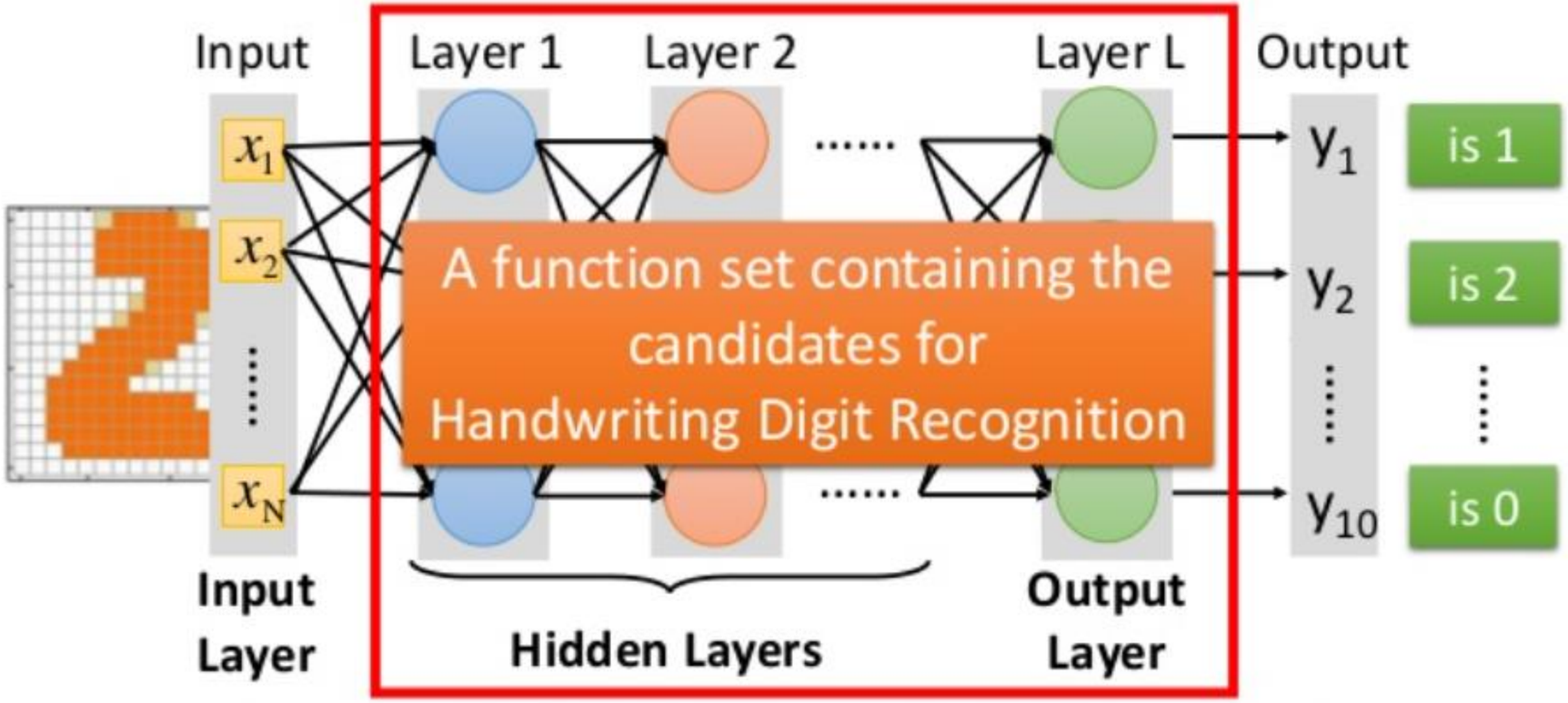


What is needed is a function ……

Input:
256-dim vector

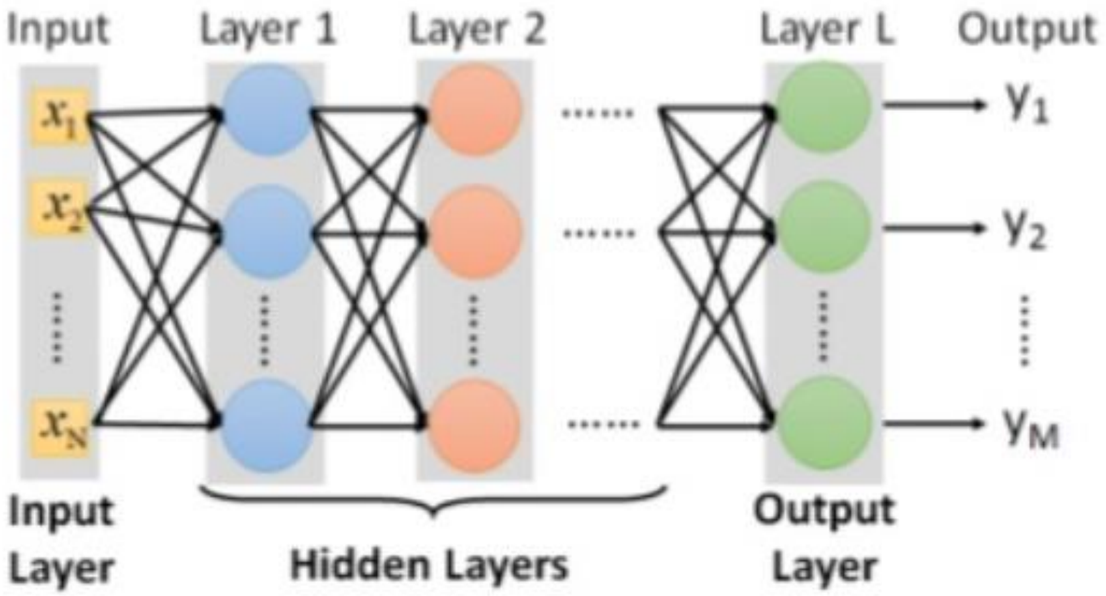
output:
10-dim vector

Example Application



You need to decide the network structure to let a good function in your function set.

FAQ



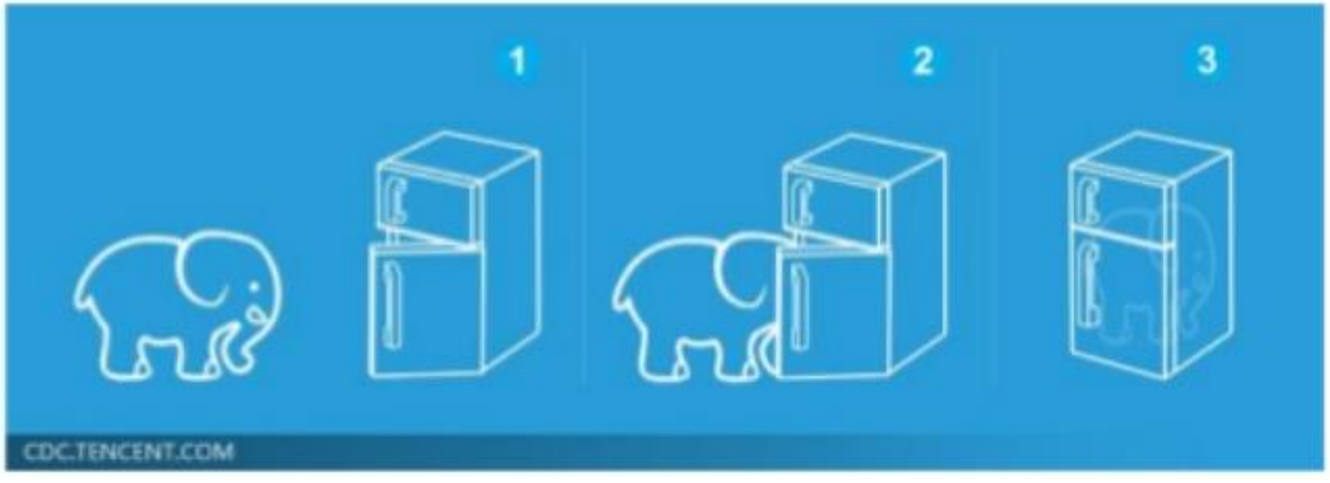
- Q: How many layers? How many neurons for each layer?

Trial and Error + Intuition

- Q: Can the structure be automatically determined?

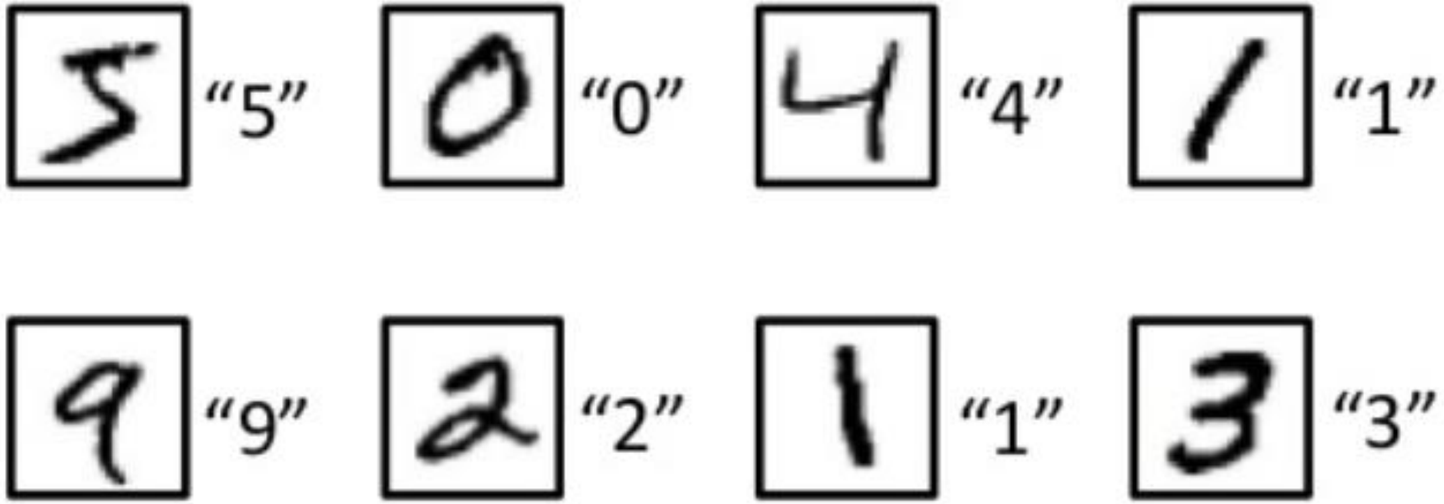
Three Steps for Deep Learning

Deep Learning is so simple ……



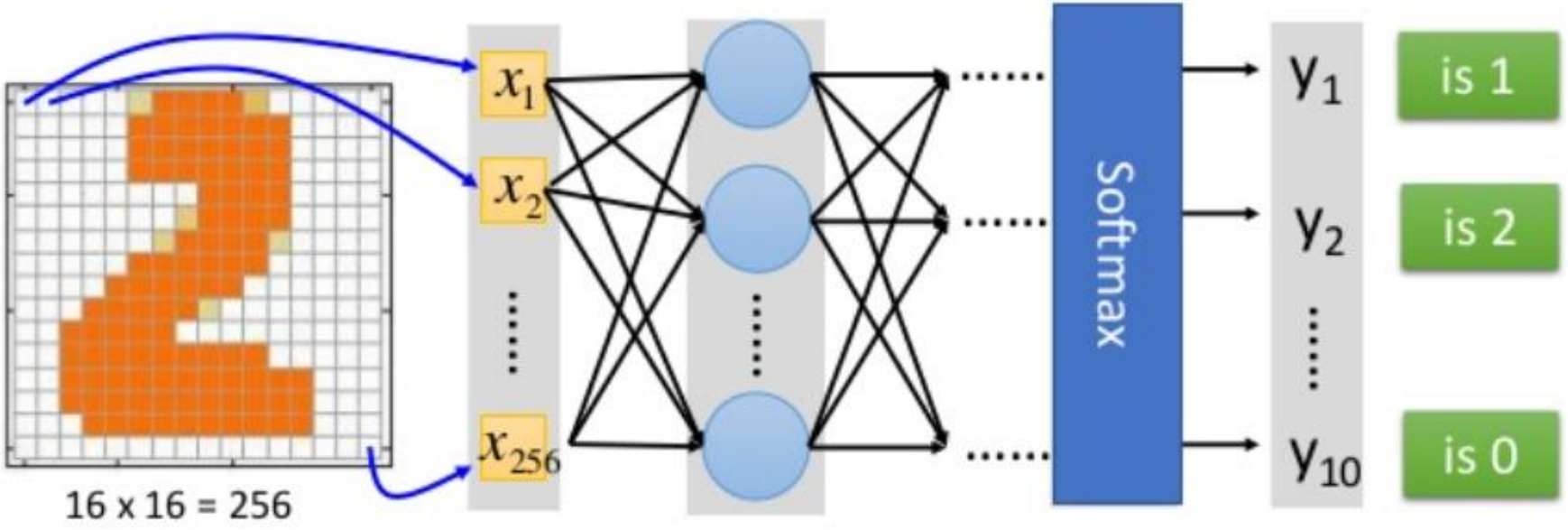
Training Data

- Preparing training data: images and their labels



The learning target is defined on the training data.

Learning Target



Ink → 1
No ink → 0

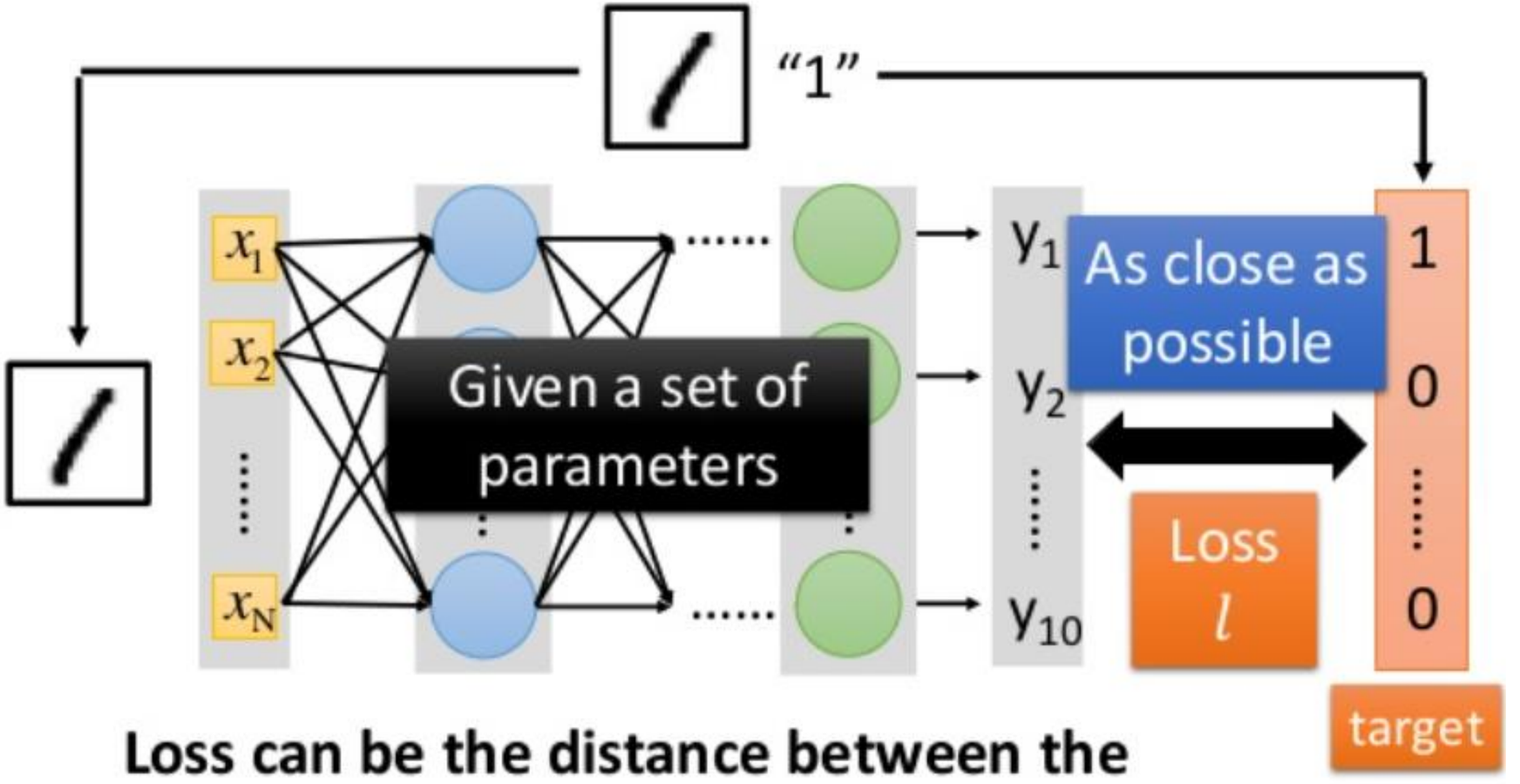
The learning target is ……

Input: → y_1 has the maximum value

Input: → y_2 has the maximum value

Loss

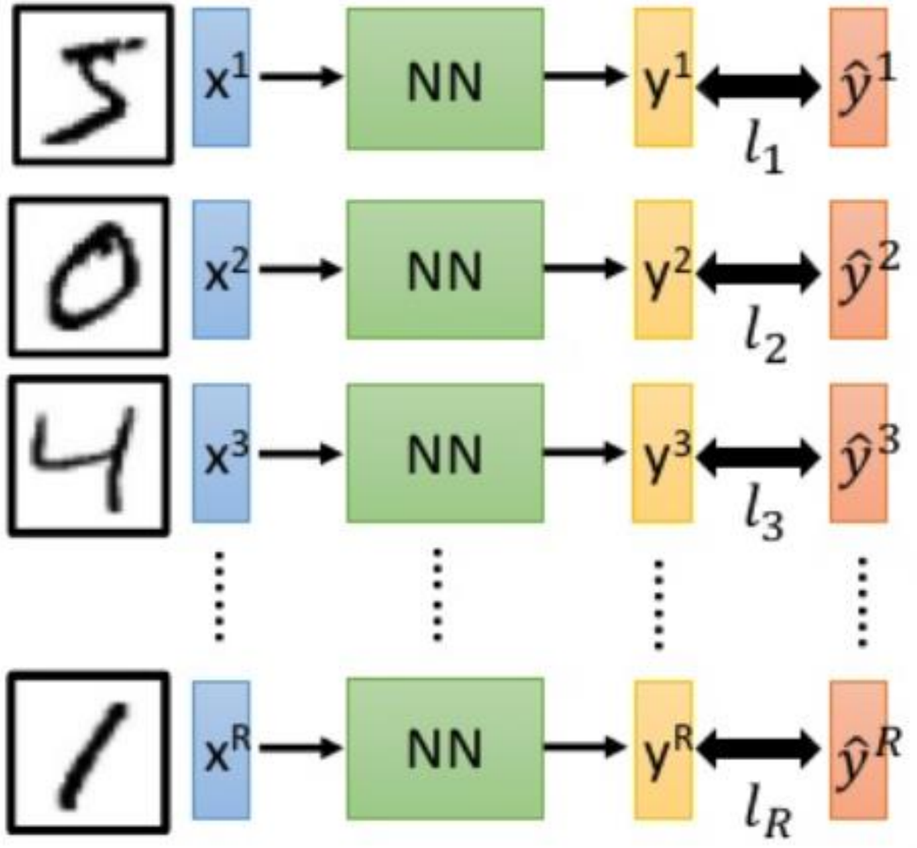
A good function should make the loss of all examples as small as possible.



Loss can be the distance between the network output and target

Total Loss

For all training data ...



Total Loss:

$$L = \sum_{r=1}^{R} l_r$$

As small as possible

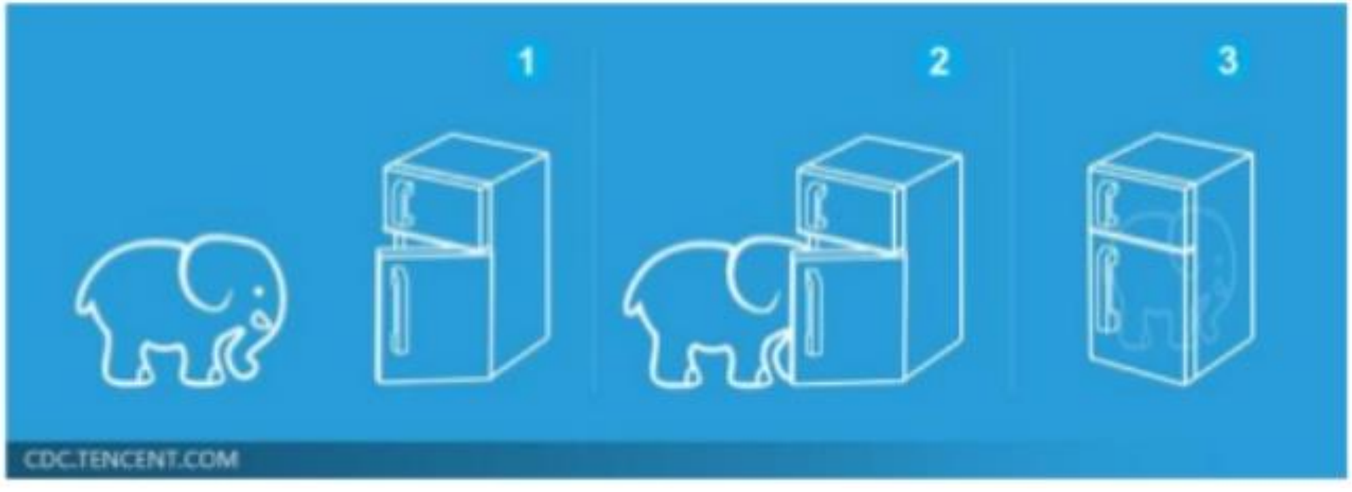
Find ***a function in function set*** that minimizes total loss L

Find ***the network parameters θ^**** that minimize total loss L

Three Steps for Deep Learning

Step 1: Neural Network → Step 2: goodness of function → Step 3: pick the best function

Deep Learning is so simple ……



How to pick the best function

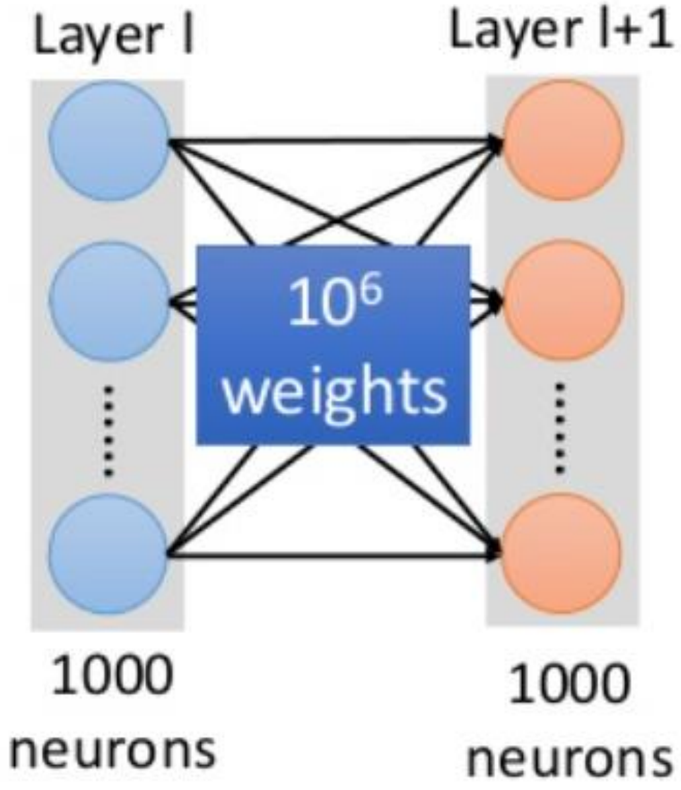
Find ***network parameters*** $\boldsymbol{\theta}^*$ that minimize total loss L

Enumerate all possible values

Network parameters $\theta = \{w_1, w_2, w_3, \cdots, b_1, b_2, b_3, \cdots\}$

Millions of parameters

E.g. speech recognition: 8 layers and 1000 neurons each layer



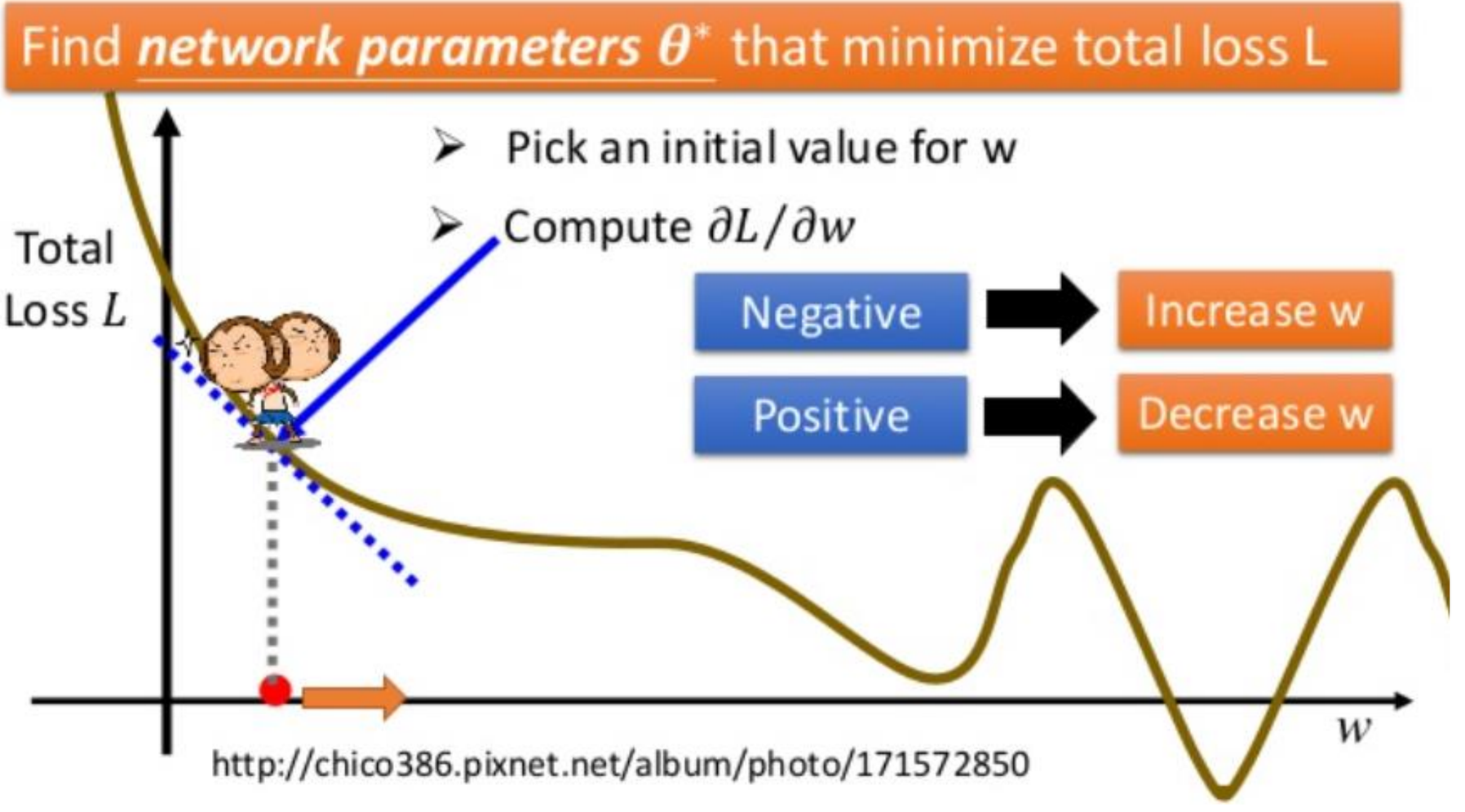
Gradient Descent

Network parameters $\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

Find ***network parameters*** $\boldsymbol{\theta}^*$ that minimize total loss L

Gradient Descent

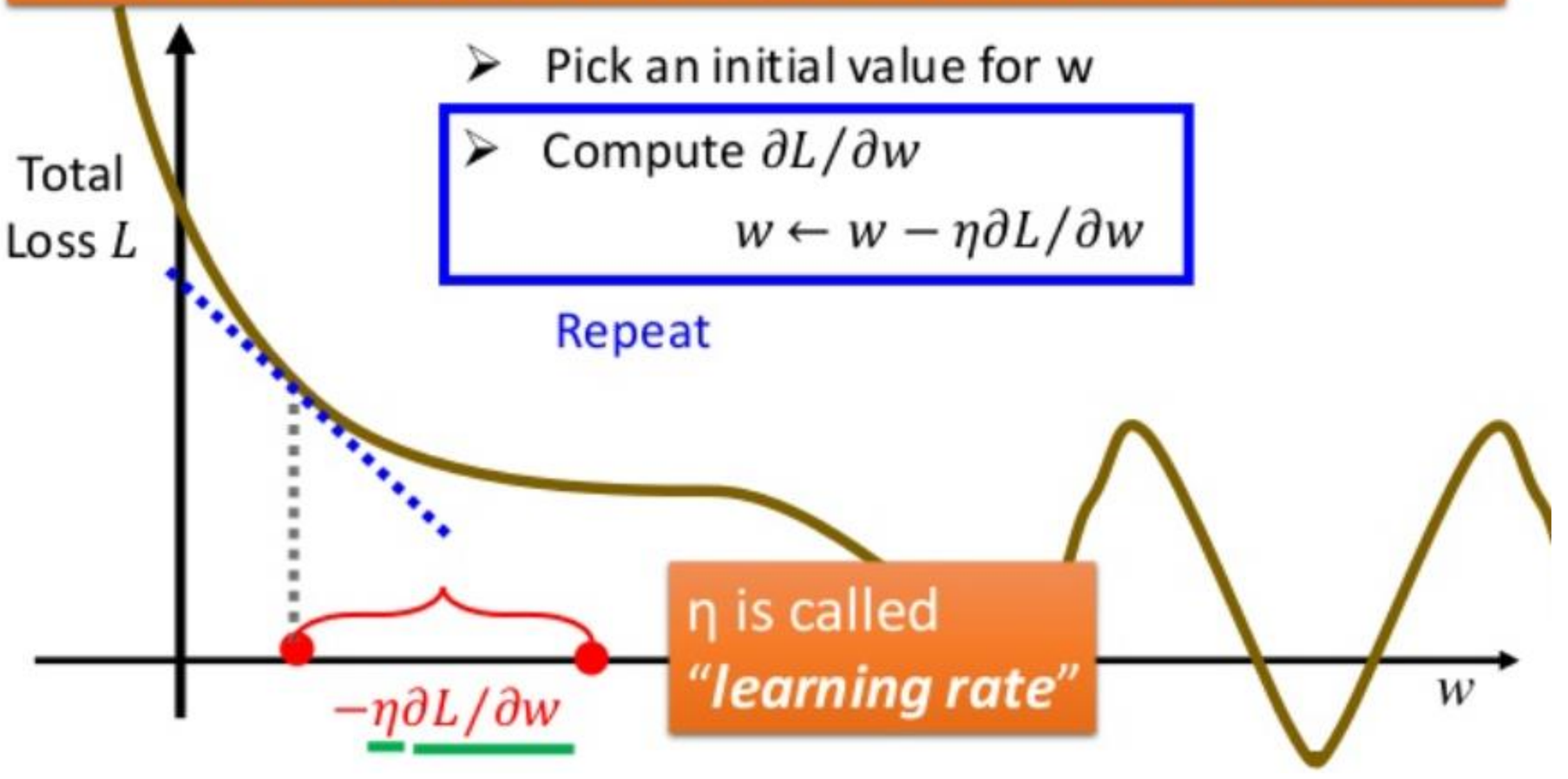
Network parameters $\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

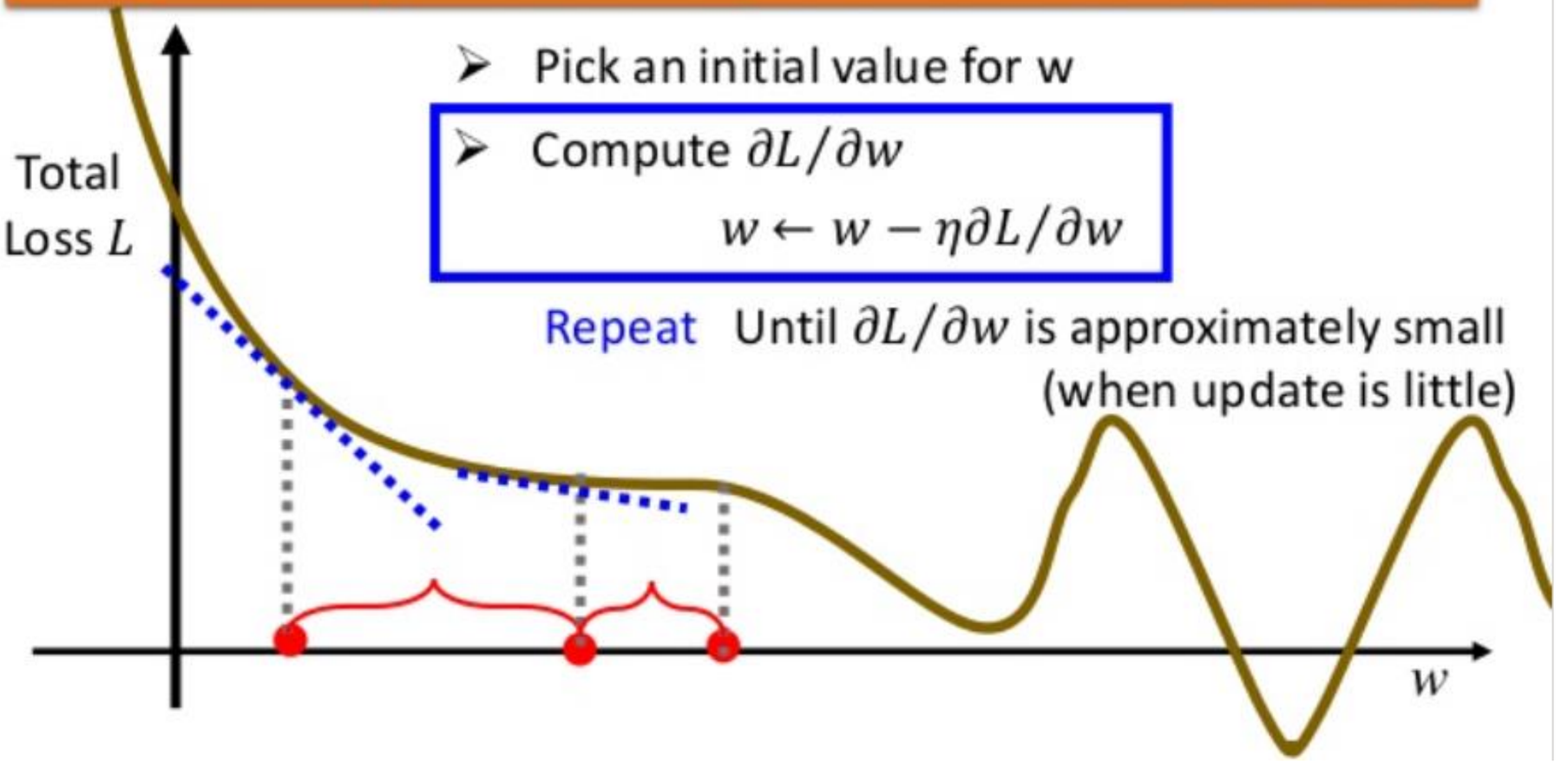
Find ***network parameters*** $\boldsymbol{\theta^*}$ that minimize total loss L



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

Find ***network parameters θ^**** that minimize total loss L

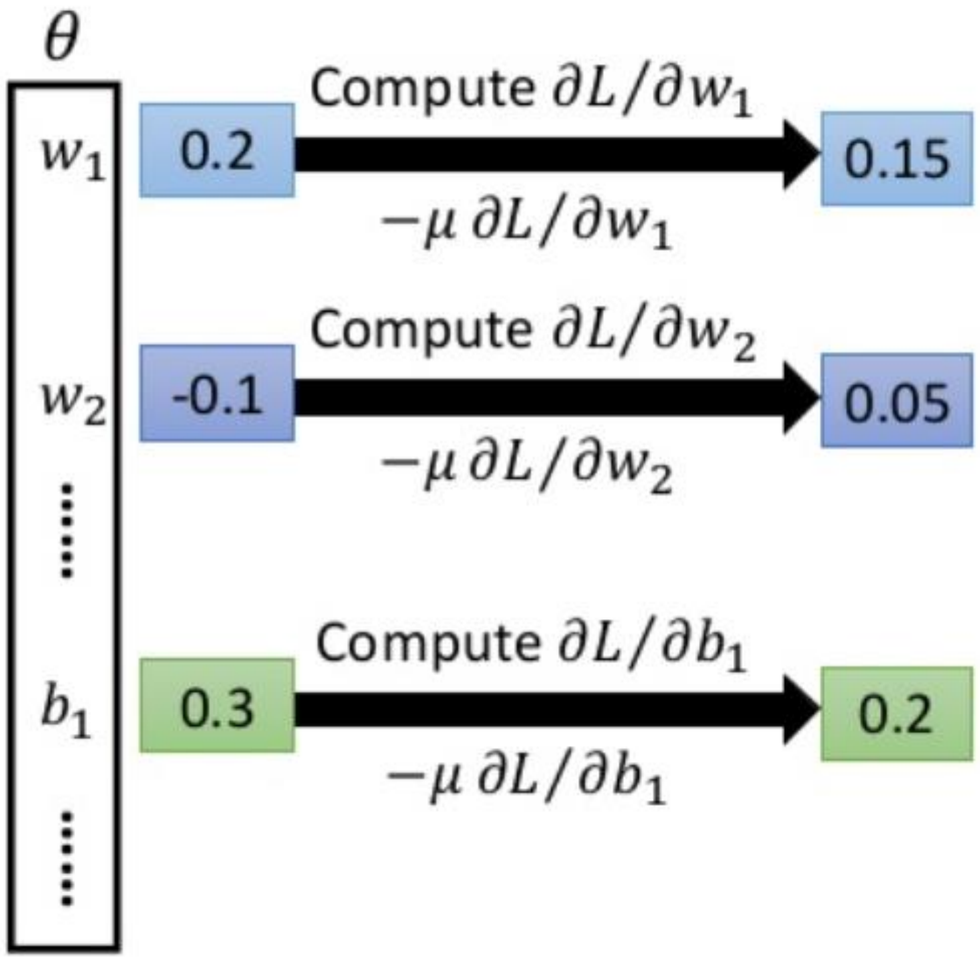


- Pick an initial value for w
- Compute $\partial L/\partial w$

$$w \leftarrow w - \eta \partial L/\partial w$$

Repeat Until $\partial L/\partial w$ is approximately small (when update is little)

Gradient Descent

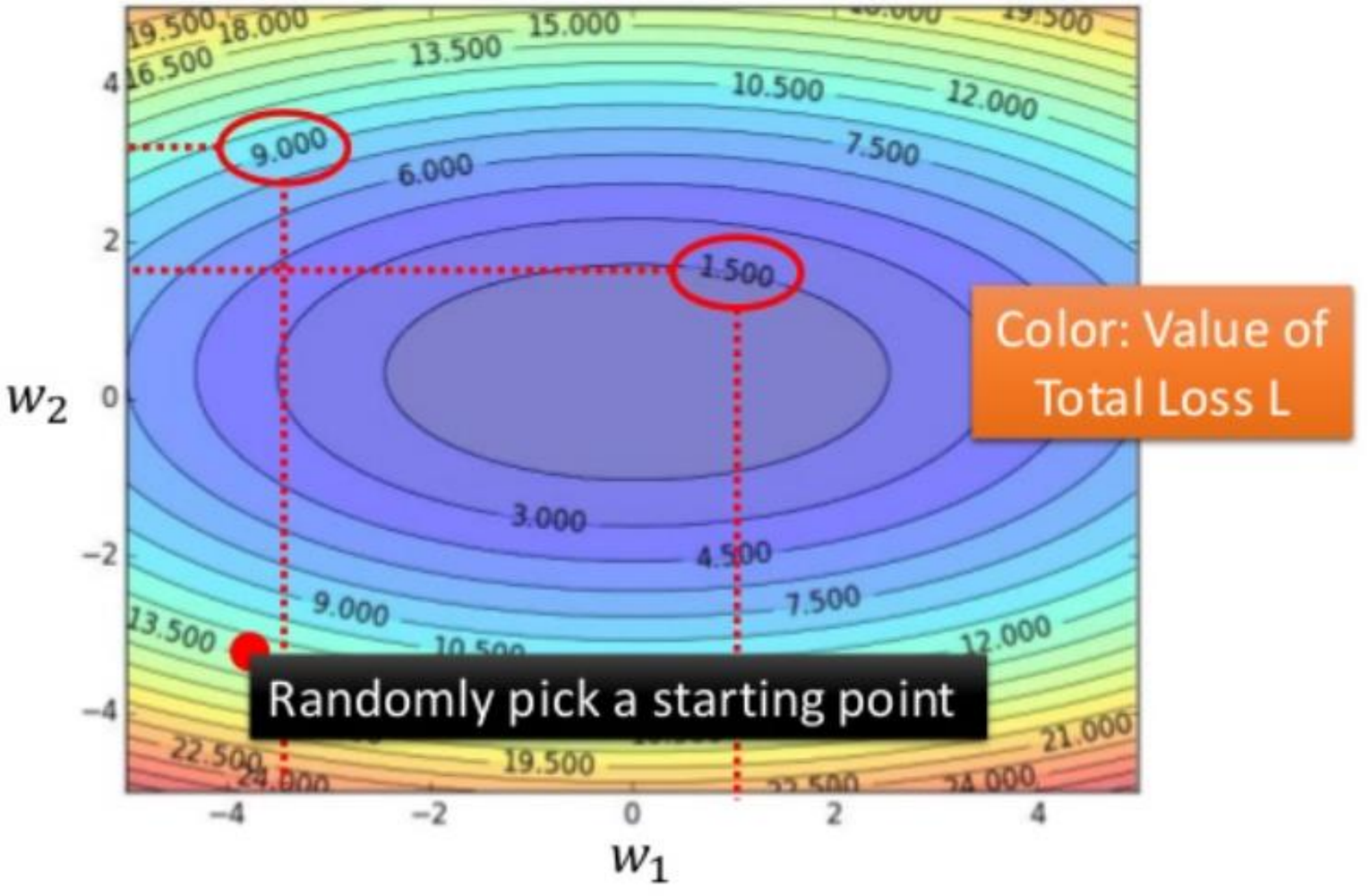


$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

gradient

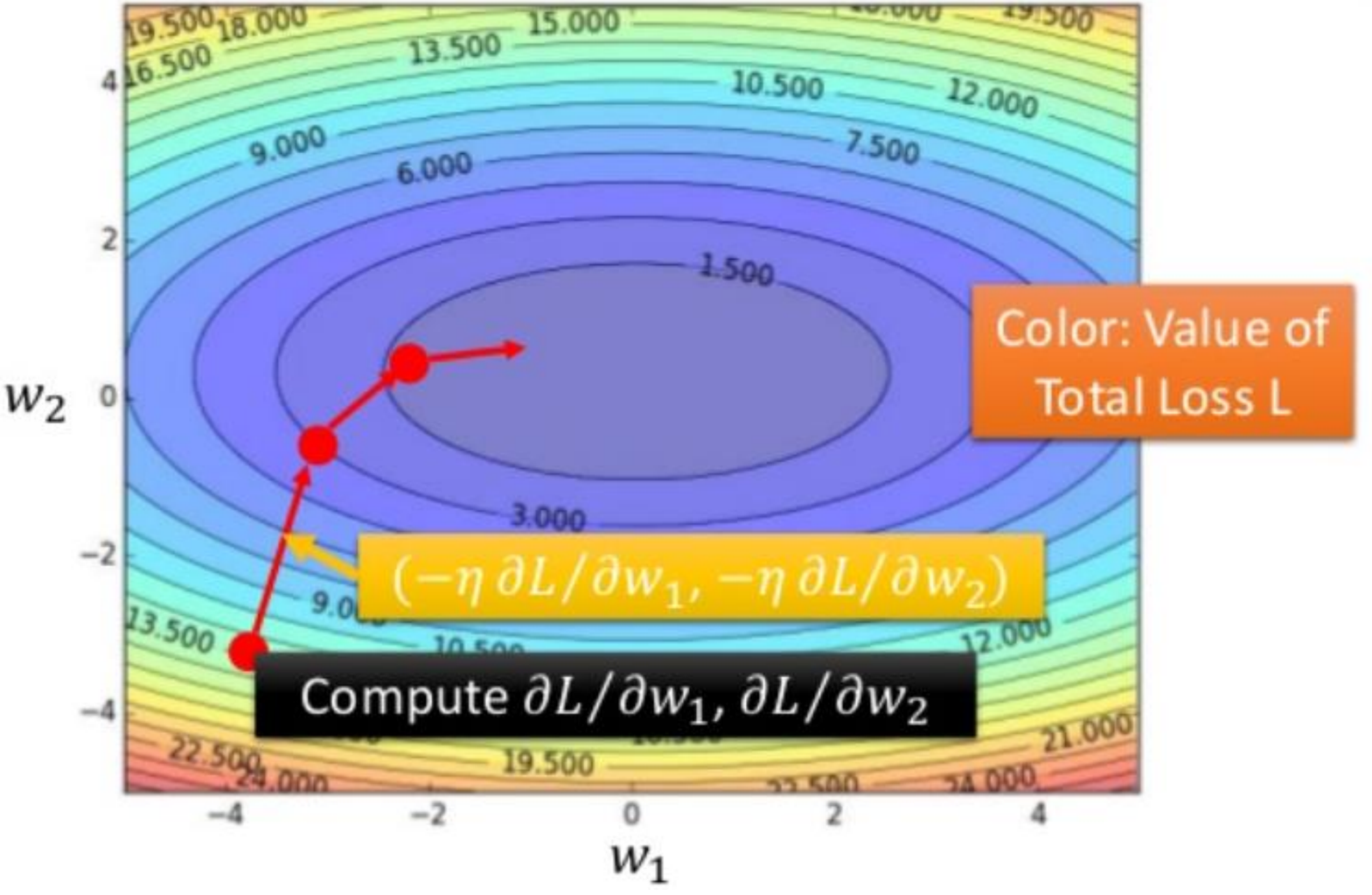
trained by error back-propagation algorithm (倒傳遞演算法).

Gradient Descent



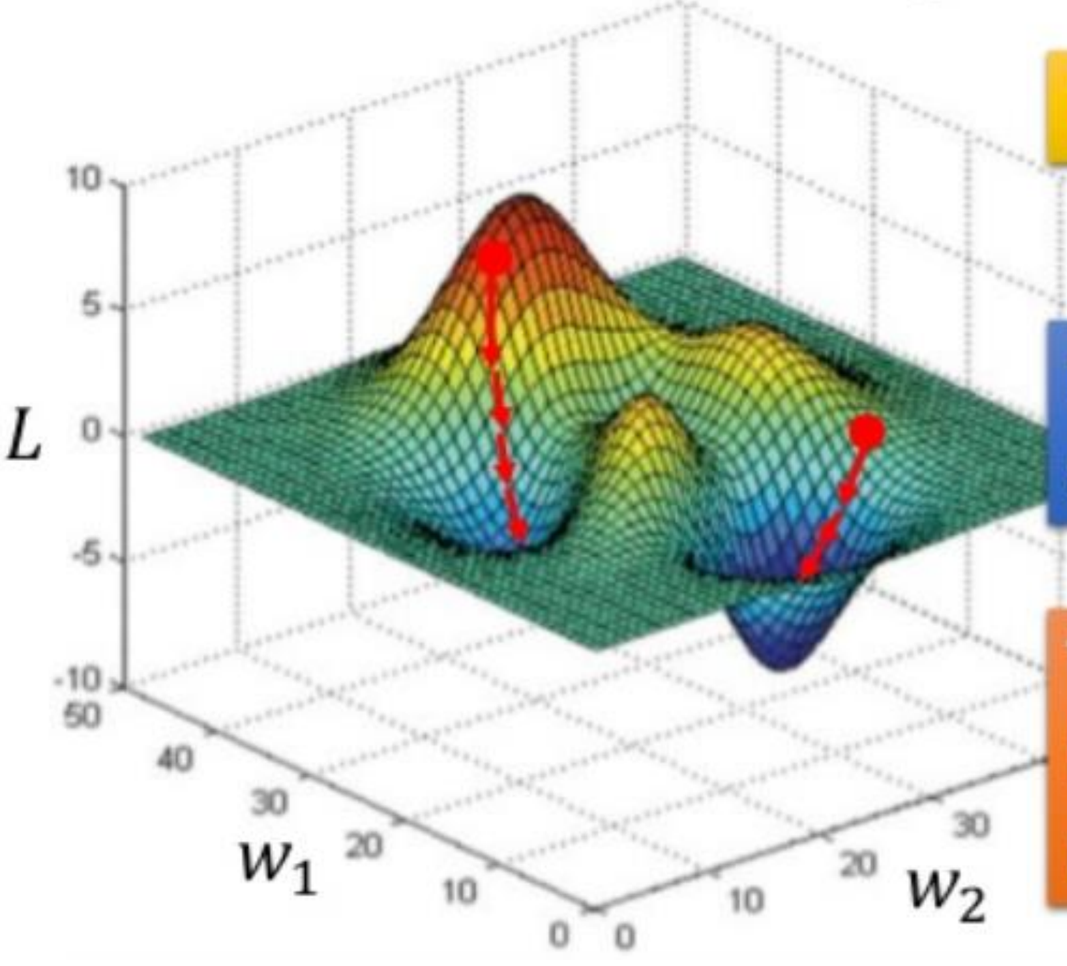
Gradient Descent

Hopfully, we would reach a minima .....



Gradient Descent - Difficulty

- Gradient descent never guarantee global minima



Different initial point

Reach different minima, so different results

There are some tips to help you avoid local minima, no guarantee.

Gradient Descent

This is the "learning" of machines in deep learning ……

➡ Even alpha go using this approach.

People image ……

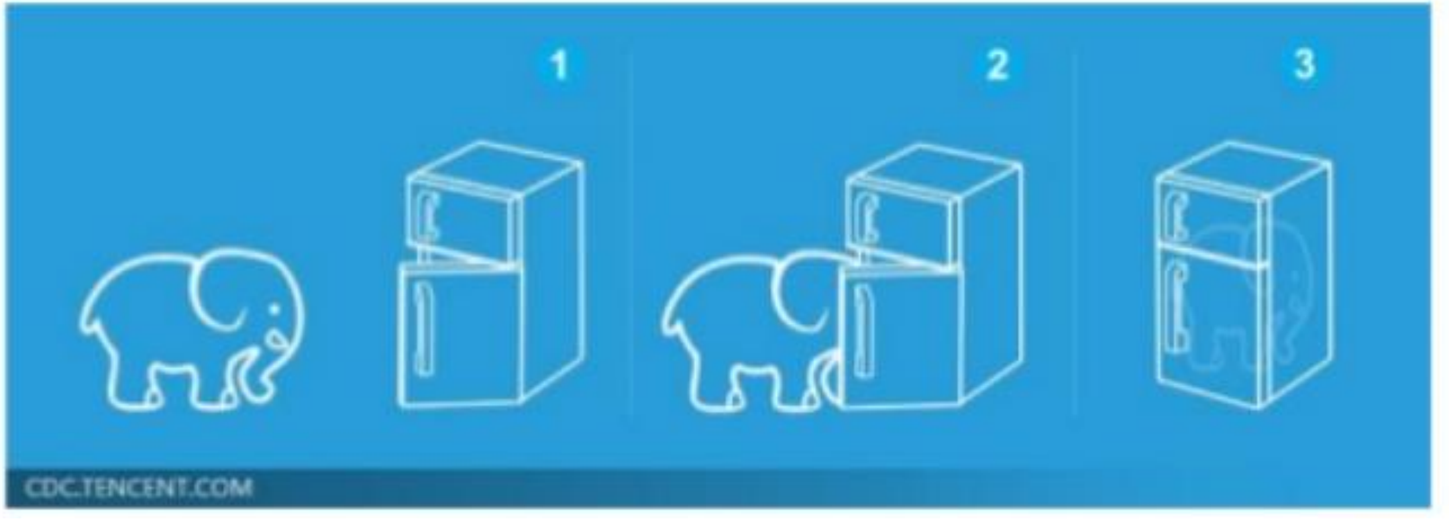
Actually …..

I hope you are not too disappointed :p

Concluding Remarks

Step 1: define a set of function → Step 2: goodness of function → Step 3: pick the best function

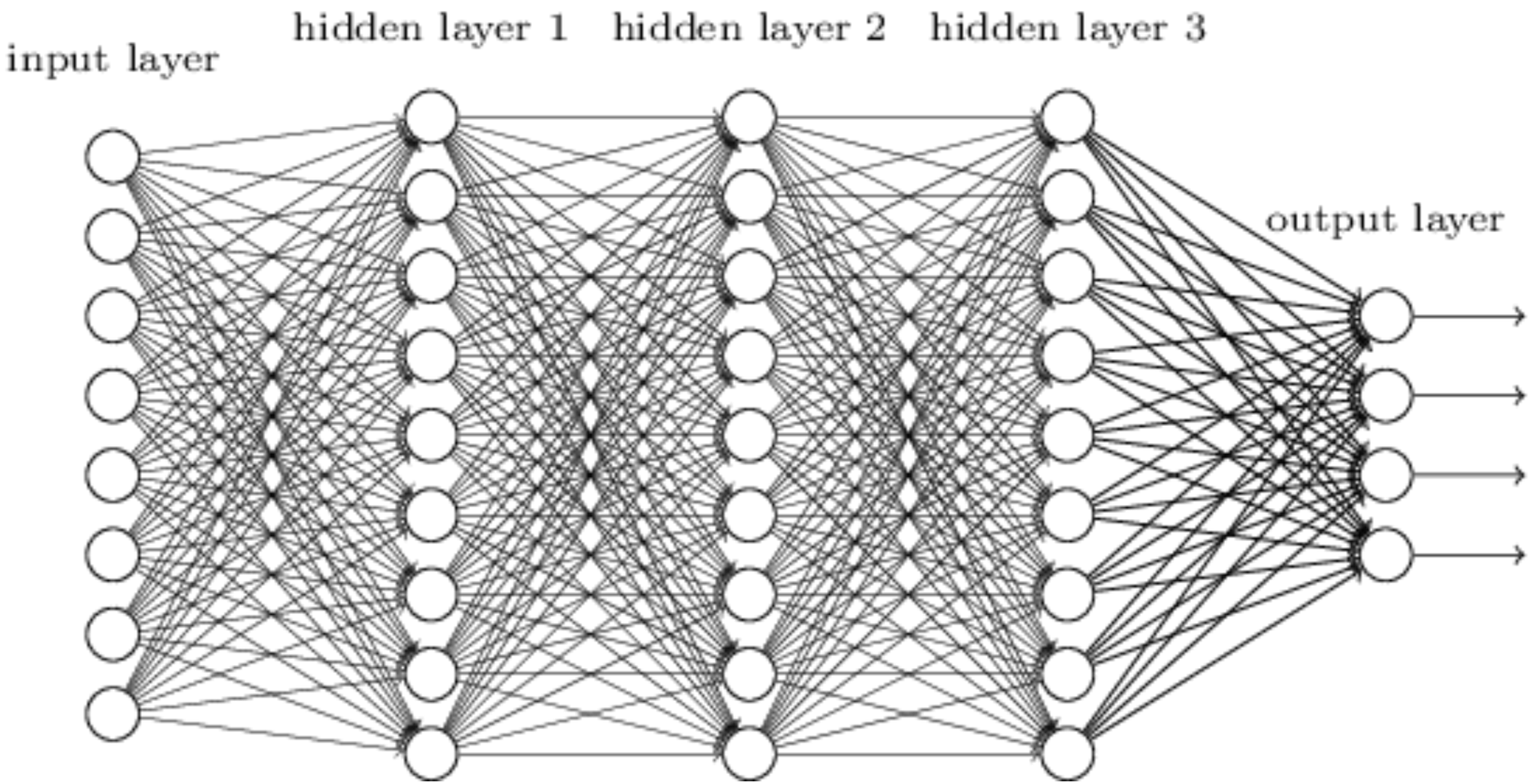
Deep Learning is so simple ……



Summary:

Training:

1. 設定參數。包括hidden layer(隱藏層)數目，隱藏層神經元數目、學習速率(learning rate)等。
2. Training 之目的在於學習在此網路設定下之權重值 (by back-propagation algorithm 倒傳遞演算法)。

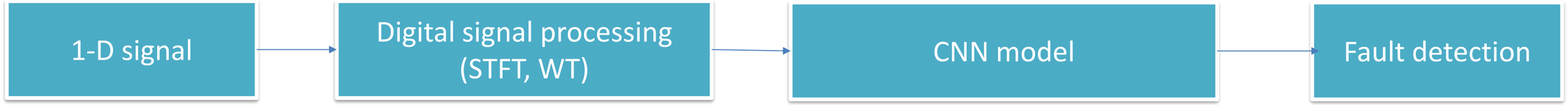


Overview of popular Deep Learning Techniques - CNN

- **Convolutional Neural Networks (CNNs) 卷積神經網路**
  1. **The convolutional layer acts as a feature extraction layer, and the fully-connected network acts as a classifiers. So, a CNN is actually a total solution for automatic pattern recognition model.**
  2. **Originally designed for 2-D images (spatial data)**
  3. **Can also applied to 1-D time-series signals (e.g., motor fault detection)**
     - **1-D CNN may not be the best for recognizing 1-D signals ➔ solution : digital signal processing techniques, such as short-time Fourier Transform (STFT) and wavelet transform (WT) can be utilized to transform 1-D signals to 2-D images (e.g., spectrograms). Then, feed them into 2-D CNNs**

Example: ML vs. CNN model for motor fault detection

Procedure of machine learning approach-based fault detection



Procedure of CNN-based for fault detection

Y.-M. Hsueh, V. R. Ittangihal, W.-B. Wu, H.-C. Chang, and C.-C. Kuo, "Fault diagnosis system for induction motors by CNN using empirical wavelet transform," *Symmetry,* vol. 11, no. 10, p. 1212, 2019.

Overview of popular Deep Learning Techniques - RNN

- **Recurrent Neural Networks (RNNs) 遞迴 or 循環神經網路**
  1. Proposed in the early 1980s, and become popular in 2010
  2. An effective DL model for recognizing 1-D time series signals.
  3. RNNs are suited for temporal sequence data, because RNNs captures dynamic time-domain characteristics by looping outputs back into the networks as inputs for subsequent steps. This action enables RNNs to preserve information across different time points.
  4. Unlike CNNs, designed for 2D spatial data, RNNs are designed for 1-D signals.
  5. Long short-term memory (LSTM) (長短期記憶模型) is a commonly-used RNN.

H. C. Cho, J. Knowles, M. S. Fadali, and K. S. Lee, "Fault detection and isolation of induction motors using recurrent neural networks and dynamic Bayesian modeling," *IEEE Transactions on Control Systems Technology,* vol. 18, no. 2, pp. 430-437, 2009.